OT-TIES: Optimal-Transport Subspaces for Interference-Resolved Merging of Low-Rank Adapters

Extended Edition with a Complete Step-by-Step Replication Record

Hilmi
https://master-hilmi.vercel.app/

June 2026

Abstract

Merging a bank of task-specialized Low-Rank Adapters (LoRAs) into a single multi-task model without retraining is an attractive route to compute-frugal model reuse. The strongest existing methods (KnOTS, Core Space, GeoMerge) succeed by two coupled ingredients: (i) they build a *shared subspace* in which the per-task updates are expressed, and (ii) inside that subspace they *resolve interference* (sign election, magnitude trimming) rather than naively averaging. The subspace, however, is invariably chosen by a singular value decomposition (SVD) of the stacked task updates, an algebraic, geometry-agnostic choice. In this project we ask whether *optimal transport* (OT) yields a better shared subspace. We make three contributions. First, we give a careful negative result: merging LoRA adapters by an OT *average* (a Wasserstein barycenter of their singular-direction distributions) plateaus at the simple-averaging floor regardless of target rank or scaling, because averaging blurs rather than resolves sign interference. Second, we introduce OT-TIES, which keeps the TIES interference-resolution operator but replaces the SVD-chosen merge subspace by the leading subspace of a *Wasserstein barycenter* of the tasks' rank-r direction distributions. On the standard 8-task CLIP ViT-B/32 LoRA benchmark, with an *identical* TIES combiner, the OT-chosen subspace beats the SVD-chosen subspace at every scaling we test (+0.7 to +1.9 normalized-accuracy points), lifting the merged model from the ~ 0.64 naive floor to 0.708. Third, we show a structural capability that the rank-fixed baselines cannot offer: because OT-TIES (and a Gromov–Wasserstein variant) operate on the full update $\Delta W = BA$ rather than a fixed-rank quotient, they natively merge a *heterogeneous-rank* adapter zoo (ranks $\{4, 8, 16\}$), losing only 1.0 point relative to the homogeneous case, a setting GeoMerge's fixed-rank quotient and Core Space's shared rank-r basis cannot enter at all. Every merge runs on CPU in minutes; the entire study was produced on free Kaggle GPUs used only for evaluation. Code, configurations, and all logs are released.

Contents

1 Introduction

Parameter-efficient fine-tuning (PEFT), and Low-Rank Adaptation (LoRA) in particular [11], has made it cheap to specialize a single pre-trained backbone into many task experts: one freezes the backbone W_0 and learns, per task t, a low-rank update $\Delta W_t = B_t A_t$ with $B_t \in \mathbb{R}^{m\times r}$, $A_t \in \mathbb{R}^{r\times n}$ and $r \ll \min(m, n)$. The resulting adapters are tiny (a few megabytes), composable, and shared in the thousands on public hubs. This abundance raises a natural question: given a *bank* of task-specialized LoRAs for the same backbone, can we *merge* them into a single model that performs all tasks well, without any further training and without access to task data?

Model merging [12, 14, 34, 35] answers "sometimes." The naive answer, average the weights, or sum the task vectors $\sum_t \Delta W_t$, degrades badly as the number of tasks grows, because independently trained updates *interfere*: they overlap in parameter space, disagree in sign, and differ wildly in magnitude [35]. The merging literature has converged on two complementary fixes. **Interference resolution** (TIES [35], DARE [36]) trims small entries, elects a per-coordinate sign by majority, and averages only the agreeing tasks. **Subspace alignment** (KnOTS [27], Core Space [20], GeoMerge [8], EpiMer [15]) first projects every task into a common low-dimensional basis, typically obtained from an SVD of the stacked updates, so that interference resolution operates on aligned coordinates. The state of the art for LoRA merging marries the two: KnOTS performs TIES *inside* an SVD-aligned space, and Core Space / GeoMerge refine the choice of basis and the averaging geometry.

A striking commonality runs through these strong methods: *the shared subspace is always chosen by linear algebra.* KnOTS concatenates the task factors and takes their SVD; Core Space builds a common alignment basis by SVD; GeoMerge averages on a fixed-rank Riemannian quotient $(\mathrm{St}\times\mathrm{SPD}\times\mathrm{St})/O(r)$ whose charts again come from SVD. None of them asks whether the *geometry of how the tasks' directions correspond to one another* should inform the basis. This is precisely the question optimal transport is built to answer: OT computes a coupling between two distributions of mass that minimizes a ground cost, and its barycenter computes the distribution that is jointly closest, in transport distance, to a family of distributions [1, 22, 30]. If we view a LoRA update as a small distribution over its singular directions, OT offers a principled, geometry-aware way to align and combine tasks.

OT is, surprisingly, almost absent from the merging literature. A recent survey of 200+ merging methods finds only a handful that use transport at all, and the one explicit OT-merging paper, OTMF [19], transports *activation* distributions to learn masks (a data-dependent, full-model procedure), not the weight-space, data-free, low-rank merge we study. The classical OT model fusion of Singh and Jaggi [26] aligns *neurons* across full $d\times d$ layers, expensive and unrelated to the low-rank structure of LoRA. The lane "merge LoRA adapters by optimal transport in the rank-r subspace" is essentially empty.

This project. We investigate whether OT yields a better merge subspace than SVD, with a deliberately controlled comparison, and we report both a negative and a positive result.

1. **A clean negative result (Section 4.1, 6.2).** The most "natural" OT merge, represent each task as a distribution over its rank-r singular directions and take their *Wasserstein barycenter*, does *not* beat naive averaging. Across target ranks $\{16, 32, 64, 128\}$ and scalings, OT-barycenter and OT-alignment both plateau at the simple-average floor (≈ 0.62 normalized accuracy), *below* tuned task arithmetic. The reason is structural and we make it precise: a transport barycenter is still an *average*, so it blurs and cancels conflicting directions; it does not perform the sign election that interference resolution requires.

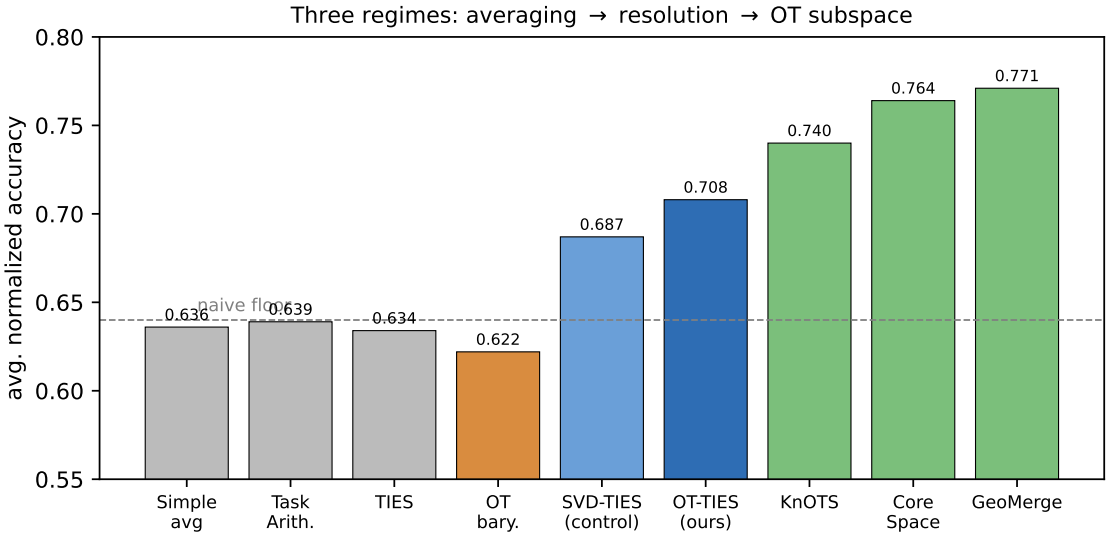

Figure 1: The three regimes we identify on the 8-task ViT-B/32 LoRA benchmark. Naive merges (gray) and the OT *average* (orange) cluster at the $\sim$0.64 floor. Interference resolution in a shared subspace lifts the merge off the floor (SVD-TIES control, light blue), and choosing that subspace by optimal transport rather than SVD adds a further, consistent gain (OT-TIES, dark blue). The published advanced methods (green) are the bar to beat; the gap to them is largely attributable to our simplified combiner, not to the OT subspace (Section 7).

2. **A positive, controlled result (Section 4.2, 6.3).** We introduce OT-TIES: keep the TIES interference-resolution operator, but choose the shared subspace as the leading left-singular subspace of the *Wasserstein barycenter* of the task direction distributions, rather than the SVD of the stacked factors (the KnOTS-style choice). Holding the TIES combiner *identical*, the OT-chosen subspace beats the SVD-chosen subspace *at every scaling tested* on the 8-task ViT-B/32 LoRA benchmark (+0.7 to +1.9 points), with a peak of 0.708 normalized accuracy versus 0.687 for the SVD control and 0.64 for the naive floor. This isolates the contribution of the *geometry of the subspace choice* from the contribution of interference resolution.
3. **A structural capability the baselines lack (Section 4.3, 6.5).** Because OT-TIES operates on the full update $\Delta W_t = B_t A_t$ rather than a fixed-rank manifold, and because we also provide a Gromov–Wasserstein (GW) variant that couples tasks by their *intra-task* geometry, our method natively merges a *heterogeneous-rank* adapter zoo (mixed ranks $\{4, 8, 16\}$). It loses only 1.0 point relative to the homogeneous-rank case, in a regime where GeoMerge's fixed-rank quotient and Core Space's shared rank-r basis cannot operate at all without ad-hoc padding.

Compute-frugality as a first-class goal. Every merge in this project is a sequence of small ($r \times r$ to $k \times k$, $r \leq 16$, $k \leq 128$) linear-algebra operations and runs on a CPU in minutes for the full ViT-B/32 model; no GPU is needed for the merge itself, only for evaluation. The complete study, reproducing five baseline configurations, two failed OT-averaging families, the OT-TIES ablation grid, and the heterogeneous-rank experiment – was carried out entirely on the free tier of Kaggle (30 GPU-hours/week, T4/P100), using the GPU solely to evaluate merged models. We view this not merely as convenience but as part of the contribution: a geometry-principled merge that is

Table 1: Summary of findings (8-task ViT-B/32 LoRA; average normalized accuracy). The contribution is the controlled OT-over-SVD subspace advantage and the heterogeneous-rank capability, both at compute-frugal CPU cost.

Finding	Evidence	Section
OT *averaging* does not beat the naive floor	0.61 to 0.63 at all ranks/scales	6.2
Interference resolution lifts off the floor	SVD-TIES 0.687, OT-TIES 0.708 vs 0.64	6.3
OT subspace > SVD subspace, identical combiner	+0.7 to +1.9 pts, every λ	6.3
Heterogeneous-rank merge (baselines cannot)	mixed $\{4, 8, 16\}$ at 0.698 (-1.0 pt)	6.5
Compute-frugal by construction	264 to 359 s CPU, full model	6.6

cheap by construction lowers the barrier for low-resource and Global-South laboratories to reuse the growing public stock of adapters. We release the full codebase, all configurations, and every run log. A complete step-by-step replication record, the probe-first methodology, every probe, the full `torch`/CUDA environment-debugging sequence, every failed kernel version and its fix, and the exact free-compute recipe, is given in Appendix A, so that a reader with only a free Kaggle account can reproduce the entire study.

2 Related Work

2.1 Model merging and task arithmetic

Weight averaging of fine-tuned models was popularized by Model Soups [34], which showed that averaging models fine-tuned with different hyperparameters can improve accuracy at no inference cost. Task Arithmetic [12] reframed merging in terms of *task vectors* $\tau_t = \theta_t - \theta_0$, observing that $\theta_0 + \lambda \sum_t \tau_t$ yields a serviceable multi-task model and that negating a task vector "forgets" a task. Git Re-Basin [2] aligns models modulo permutation symmetries before averaging, an early signal that *alignment* matters. A comprehensive taxonomy appears in the deep model fusion survey of Li et al. [14].

2.2 Interference resolution

TIES-Merging [35] identified three interference modes when summing task vectors – redundant parameters, sign conflicts, and magnitude disparities, and resolves them by (i) *trimming* all but the top-magnitude entries of each task vector, (ii) electing a per-parameter sign by the sign of the aggregate, and (iii) averaging only the tasks that agree with the elected sign. DARE [36] shows that delta parameters are highly redundant and can be randomly dropped and rescaled before merging. These operators are the empirical backbone of strong merges; our OT-TIES re-uses the TIES operator unchanged and varies only the subspace it acts in.

2.3 Subspace / SVD alignment for LoRA merging

LoRA updates are explicitly low rank, which makes the choice of a *shared* subspace both natural and consequential. KnOTS [27] aligns the per-task updates in a common space obtained by SVD of the concatenated factors, then applies TIES; it substantially improves over space-agnostic TIES. Core Space [20] merges low-rank adaptations inside a common alignment basis, preserving low-rank efficiency while improving accuracy, and reports state-of-the-art results on vision and language LoRA benchmarks. Decouple-and-Orthogonalize [42] addresses the large magnitude disparities

specific to LoRA factors. IterIS [6] refines an alignment by iterative inference-solving. A parallel line constrains the LoRA subspace *before* or *during* fine-tuning to reduce later interference: LoRI [39] freezes A as a random projection and sparsifies B with task masks; Zhang and Zhou [38] pre-allocate orthogonal subspaces; LEGO [41] clusters rank-one components for modular reuse. All of these obtain the shared basis algebraically (SVD, orthogonality, random projection). We instead obtain it by optimal transport and compare head-to-head with the SVD choice under an identical combiner.

2.4 The geometry of merging

A recent and influential thread recasts merging as averaging on a *manifold*. GeoMerge [8] formulates merging as a Fréchet average under an explicit geometry and, with a high-rank lift, sets the current state of the art on the 8-task ViT-B/32 LoRA benchmark (77.1% normalized accuracy). EpiMer [15] computes a Fréchet mean on a Riemannian manifold restricted to the task-vector subspace using a projected Hessian. On the optimization side, Stiefel-LoRA [21], RiemannLoRA [4], and Riemannian preconditioned LoRA [37] bring Riemannian geometry to LoRA *training*; AlignGuard-LoRA [9] uses Fisher-guided decomposition to preserve alignment. These methods share a feature that bounds their applicability: their manifolds are *rank-fixed* (the quotient $(\mathrm{St} \times \mathrm{SPD} \times \mathrm{St})/O(r)$ and the shared rank-r basis both require all adapters to have the same rank r). Our heterogeneous-rank experiment (Section 6.5) targets exactly the regime these methods cannot enter. Concretely, EpiMer [15] restricts the Fréchet mean to the task-vector subspace and exploits a projected Hessian for curvature, while GeoMerge [8] parameterizes the merged adapter on the product manifold of left/right Stiefel factors and an SPD core, quotiented by the $O(r)$ rotation gauge, a construction that is only well defined when all adapters share the rank r that fixes the Stiefel and SPD dimensions. The Riemannian-*training* line [4, 21, 37] likewise pins the rank during optimization. None of these can accept a bank of adapters with ranks $\{4, 8, 16\}$ without first padding or projecting to a common r (which either fabricates or discards signal). Our method needs no such homogenization because it works on $\Delta W_t = B_t A_t$ and a fixed ambient dimension k.

2.5 Optimal transport in model merging

OT is mature in machine learning, Sinkhorn's entropic regularization [7] made it differentiable and fast; Wasserstein barycenters [1] average distributions geometrically; Gromov–Wasserstein [17, 23] couples distributions across *incomparable* spaces; the POT library [10] provides solvers. Yet in merging, OT is rare. Singh and Jaggi [26] fuse models by transporting *neurons* across full layers, a $d \times d$ problem unrelated to low-rank structure. OTMF [19] transports *activation* distributions to learn fusion masks in a continual setting, requiring data forward passes. To our knowledge no prior work uses OT to choose the *weight-space, data-free, low-rank* merge subspace, which is the gap we fill.

2.6 PEFT, LoRA optimization, and continual adaptation

A broad literature improves *how* LoRAs are trained and structured, which is complementary to (and upstream of) merging. On optimization, LoRA-Pro [33] corrects the gradient geometry of the low-rank factors, LoRA-GA [31] aligns the initial update with the full-fine-tuning gradient, and the Riemannian-training family [4, 21, 37] optimizes the factors on Stiefel/quotient manifolds to remove the scale/rotation ambiguity intrinsic to BA. On structure and efficiency, Uni-LoRA [13] reinterprets many LoRA variants as a single global subspace projection, LISA [18] replaces low-rank updates with layerwise importance sampling, LoSiA [32] localizes high-rank subnets, and a stronger

Table 2: Where the main data-free LoRA-merge methods sit. "Subspace" = how the shared basis is chosen; "Combine" = averaging vs. interference resolution; "Het. rank" = can it merge adapters of *different* ranks without padding; "CPU merge" = the merge step needs no GPU.

Method	Subspace	Combine	Het. rank	CPU merge
Task Arithmetic [12]	none (ambient)	scaled sum	✓	✓
TIES [35]	none (ambient)	resolution	✓	✓
KnOTS [27]	SVD of stacked factors	resolution (TIES)	✓†	✓
Core Space [20]	SVD alignment basis	resolution	✗ (rank-r)	partly
GeoMerge [8]	fixed-rank quotient	Fréchet *average*	✗ (rank-r)	✗ (Riem. loop)
OT-TIES (ours)	**OT barycenter**	resolution (TIES)	✓	✓
GW (ours)	GW (intra-task)	resolution	✓	✓

† KnOTS's stacked-SVD basis admits mixed column counts in principle but is not evaluated in that regime; the geometric methods (Core/GeoMerge) are rank-fixed by construction.

mixture-of-LoRA-experts [28] preconditions expert gradients. The *asymmetry* of the two factors, A extracts features, B projects them, was made explicit by Zhu et al. [43], motivating our use of paired (left $\times$ right) ground costs rather than a single-factor cost. In continual settings, C-LoRA [40] and the perturb-then-merge framework of Qiu et al. [24] fold merging into the learning loop. RDP-LoRA [5] and CopRA [44] use representational geometry and progressive training respectively to choose where/how to adapt. Our work is orthogonal to all of these: it takes *already-trained* adapters (of possibly heterogeneous rank) and asks only how best to combine them, data-free and training-free.

2.7 Positioning

In one sentence: prior strong LoRA merges resolve interference inside an *SVD-chosen* shared subspace; we keep the resolver and replace the subspace by an *OT-chosen* one, and we additionally remove the rank-homogeneity assumption that the geometric state of the art structurally requires. Table 2 places the main methods on the axes that matter for this project.

3 Background and Problem Setup

Table 3: Notation.

Symbol	Meaning
$W_0 \in \mathbb{R}^{m\times n}$	frozen backbone weight at a layer
$A_t \in \mathbb{R}^{r_t\times n}$, $B_t \in \mathbb{R}^{m\times r_t}$	LoRA down/up factors of task t
$s_t = \alpha_t/r_t$	LoRA scaling of task t
$\Delta W_t = s_t B_t A_t$	task-t update; $\Delta W^\star$ the merged update
U_t, Σ_t, V_t	thin SVD factors of ΔW_t (Stiefel, diagonal, Stiefel)
$\mu_t = \sum_k p_{t,k}\delta_{(u_{t,k},v_{t,k})}$	task-t direction distribution, $p_{t,k} \propto \sigma_{t,k}$
T	number of tasks/adapters; r common rank when homogeneous
k	shared-subspace dimension ($k \le \sum_t r_t$)
κ	TIES keep fraction; λ scale; ε Sinkhorn reg.
$C \in \mathbb{R}^{r_i\times r_j}$	ground cost; $P \in \Pi(\boldsymbol{p}, \boldsymbol{q})$ transport plan
ν	Wasserstein barycenter of $\{\mu_t\}$; U its leading subspace
NA	average normalized accuracy (Eq. (2))

3.1 LoRA updates and the merging problem

Fix a backbone with frozen weight $W_0 \in \mathbb{R}^{m\times n}$ at a given layer. A LoRA adapter for task t adds

$$\Delta W_t \;=\; s_t\, B_t A_t, \qquad B_t \in \mathbb{R}^{m\times r_t},\; A_t \in \mathbb{R}^{r_t\times n},\; s_t = \frac{\alpha_t}{r_t}, \tag{1}$$

where s_t is the LoRA scaling (α_t is the configured `lora_alpha`). Given T adapters $\{(A_t, B_t, s_t)\}_{t=1}^{T}$ for the *same* backbone, a layer-wise merge produces a single update $\Delta W^\star$ so that $W_0 + \Delta W^\star$ performs all T tasks. The merge is *data-free* (it sees only the adapter weights) and *training-free*. We measure quality by the average *normalized accuracy*

$$\mathrm{NA} \;=\; \frac{1}{T}\sum_{t=1}^{T} \frac{\mathrm{acc}_t(\mathrm{merged})}{\mathrm{acc}_t(\mathrm{specialist}_t)}, \tag{2}$$

the standard single number for this benchmark, where $\mathrm{acc}_t(\mathrm{specialist}_t)$ is the accuracy of the t-th adapter on its own task.

3.2 Task vectors, averaging, and TIES

Stacking layers, the task vector is $\tau_t = \mathrm{vec}(\Delta W_t)$. Simple averaging returns $\frac{1}{T}\sum_t \tau_t$; task arithmetic returns $\lambda \sum_t \tau_t$. TIES [35] replaces the sum by an interference-resolved combine. Given per-task coordinate values $\{s_t\}_{t=1}^{T}$ at a coordinate, TIES (i) trims each task to a top-κ fraction by magnitude, (ii) elects the sign $\gamma = \mathrm{sign}(\sum_t s_t)$, and (iii) returns $\lambda \cdot \frac{\sum_{t:\,\mathrm{sign}(s_t)=\gamma} s_t}{|\{t:\mathrm{sign}(s_t)=\gamma\}|}$, i.e. a *disjoint mean* over the tasks agreeing with the elected sign. The sign election is the crucial nonlinearity: it is *not* an average, and it is exactly what naive and OT-average merges lack.

3.3 Two views of merging: Fréchet mean vs. interference resolution

It clarifies the landscape to name the two philosophies that the literature pursues. The *averaging* view seeks a central point: given a geometry g on parameter (or task-vector) space, return the Fréchet mean $\Delta W^\star = \arg\min_x \sum_t \lambda_t\, d_g(x, \Delta W_t)^2$. Simple averaging is the Euclidean instance; GeoMerge [8] and EpiMer [15] use richer manifold geometries; an OT barycenter is the instance where d_g is a Wasserstein distance between direction distributions. The *resolution* view does not seek a central point at all: it treats the per-coordinate values as votes and applies a nonlinear, sign-aware aggregator (TIES, DARE). Our Proposition 1 draws the line between them sharply: *no* member of the averaging family, however clever its geometry, can recover a coordinate on which two equally strong tasks disagree in sign, because the minimizer of a sum of squared distances to $+\sigma w$ and $-\sigma w$ is 0. The resolution family can. This is why, empirically, moving from any average (including the OT barycenter) to a resolver is worth $\sim$7 normalized-accuracy points, dwarfing the differences *within* the averaging family. Our method is therefore deliberately a *resolver*; the role of optimal transport is not to average better but to supply the resolver with a better coordinate system.

3.4 Singular directions of a LoRA update

We will repeatedly use the (thin) SVD of an update. Writing $\Delta W_t = U_t \Sigma_t V_t^\top$ with $U_t \in \mathbb{R}^{m\times r_t}$, $V_t \in \mathbb{R}^{n\times r_t}$ on Stiefel manifolds and $\Sigma_t = \mathrm{diag}(\sigma_{t,1}, \ldots, \sigma_{t,r_t})$, we view each task as an empirical distribution over its r_t singular directions,

$$\mu_t \;=\; \sum_{k=1}^{r_t} p_{t,k}\, \delta_{(u_{t,k},\, v_{t,k})}, \qquad p_{t,k} = \frac{\sigma_{t,k}}{\sum_j \sigma_{t,j}}, \tag{3}$$

a mass-$p_{t,k}$ atom at the paired left/right direction $(u_{t,k}, v_{t,k})$. Crucially, because $\Delta W_t = s_t B_t A_t$ has rank at most r_t, this SVD is computed in the $r_t \times r_t$ space via two thin QR factorizations of B_t and $A_t^\top$ followed by an $r_t \times r_t$ SVD; the full $m \times n$ matrix is never formed. All transport problems below are therefore $r_i \times r_j$ or $k \times k$ with $r \le 16$, $k \le 128$, microseconds to milliseconds on a CPU.

Lemma 1 (Exact rank-r SVD without densification)**.** *Let $\Delta W = s\,BA$ with $B \in \mathbb{R}^{m\times r}$, $A \in \mathbb{R}^{r\times n}$. Let $B = Q_B R_B$ and $A^\top = Q_A R_A$ be thin QR factorizations ($Q_B \in \mathbb{R}^{m\times r}$, $Q_A \in \mathbb{R}^{n\times r}$), and let $s\,R_B R_A^\top = \tilde{U}\tilde{\Sigma}\tilde{V}^\top$ be the (small) $r \times r$ SVD. Then $U = Q_B\tilde{U}$, $\Sigma = \tilde{\Sigma}$, $V = Q_A\tilde{V}$ is an exact thin SVD of ΔW, computed in $O((m+n)r^2 + r^3)$ time and $O((m+n)r)$ memory, never forming the $m \times n$ product.*

Proof. $\Delta W = sBA = s(Q_B R_B)(Q_A R_A)^\top = Q_B\,(sR_B R_A^\top)\,Q_A^\top = Q_B(\tilde{U}\tilde{\Sigma}\tilde{V}^\top)Q_A^\top = (Q_B\tilde{U})\tilde{\Sigma}(Q_A\tilde{V})^\top$. Since $Q_B, \tilde{U}$ have orthonormal columns, so does $Q_B\tilde{U}$ (likewise $Q_A\tilde{V}$); $\tilde{\Sigma}$ is diagonal nonnegative. Hence the right-hand side is a thin SVD of ΔW. The QRs cost $O(mr^2)$ and $O(nr^2)$, the small SVD $O(r^3)$, and no intermediate is larger than $m \times r$ or $n \times r$. □

Lemma 1 is the workhorse that keeps every operation in the rank-r (or barycenter rank-k) regime. We also fix the sign ambiguity of each singular pair by forcing the largest-magnitude entry of every left vector positive and flipping the paired right vector, which makes the whole pipeline deterministic.

3.5 Optimal transport, barycenters, Gromov–Wasserstein

Given two discrete distributions with masses $\boldsymbol{p} \in \Delta^{r_i}$, $\boldsymbol{q} \in \Delta^{r_j}$ and a ground cost $C \in \mathbb{R}^{r_i \times r_j}$, the entropic OT problem [7] is

$$P^\star = \operatorname*{arg\,min}_{P \in \Pi(\boldsymbol{p},\boldsymbol{q})} \langle P, C\rangle - \varepsilon H(P), \qquad \Pi(\boldsymbol{p},\boldsymbol{q}) = \{P \ge 0 : P\mathbf{1} = \boldsymbol{p},\ P^\top\mathbf{1} = \boldsymbol{q}\}, \tag{4}$$

with H the entropy and $\varepsilon \ge 0$; as $\varepsilon \to 0$ one recovers the exact (EMD) coupling. The *Wasserstein barycenter* [1] of $\{\mu_t\}$ with weights λ_t is the distribution ν minimizing $\sum_t \lambda_t W^2(\nu, \mu_t)$. When supports are free, it is computed by alternating (i) transport from the current ν to each μ_t and (ii) a barycentric update of ν's atoms. Gromov–Wasserstein (GW) [17, 23] couples two distributions living in *incomparable* spaces by matching their *intra*-distribution distance matrices $D^{(i)}, D^{(j)}$:

$$\min_{P \in \Pi(\boldsymbol{p},\boldsymbol{q})} \sum_{k,k',l,l'} \big(D^{(i)}_{k,k'} - D^{(j)}_{l,l'}\big)^2 P_{k,l} P_{k',l'} - \varepsilon H(P). \tag{5}$$

GW needs no shared ambient frame, which is exactly what lets it relate adapters of *different ranks*.

The entropic problem (4) is solved by Sinkhorn's matrix-scaling iteration [7]: with kernel $K = \exp(-C/\varepsilon)$ one alternates $\boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (K\boldsymbol{v})$, $\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (K^\top\boldsymbol{u})$ and returns $P = \operatorname{diag}(\boldsymbol{u}) K \operatorname{diag}(\boldsymbol{v})$, converging linearly in the Hilbert metric. Because our C is $r_i \times r_j$ with $r \le 16$ (or $k \times r$ with $k \le 128$), a handful of iterations suffice; for very small problems we fall back to the exact network-flow (EMD) solver, which is microseconds at this size. The free-support Wasserstein barycenter is computed by the alternating scheme of Agueh and Carlier [1], Peyré and Cuturi [22]: hold the support ν fixed and transport to each μ_t; then update each atom of ν as the λ-weighted barycentric image of its transported mass; iterate. We keep this update *factored*, the accumulated left/right factors are stacked and re-orthogonalized by the thin-SVD of Lemma 1, so a barycenter over eight rank-16 tasks never forms a dense 768×768 matrix until the final reconstruction.

Remark 1 (Why a single ground cost is principled here)**.** *The asymmetry result of Zhu et al. [43] shows the left factor B (which spans the output directions U) carries the task-discriminative content, while A (spanning V) acts more like a fixed feature reader. This motivates two of our design choices: (i) the* paired *cost C^{paired} couples left* and *right collinearity so that a match must agree on both the "what" and the "where"; (ii) our reconstruction is* one-sided *in the left basis U, which we find empirically dominates the two-sided variant (Section 6.3), consistent with the output directions being the load-bearing ones.*

3.6 Ground costs between direction distributions

For two tasks with Stiefel factors (U_i, V_i), (U_j, V_j) we use sign-invariant collinearity costs (LoRA singular vectors are defined up to a sign):

$$C^{\text{paired}}_{k,l} = 1 - |\langle u_{i,k}, u_{j,l}\rangle|\,|\langle v_{i,k}, v_{j,l}\rangle|, \qquad C^{\text{left}}_{k,l} = 1 - |\langle u_{i,k}, u_{j,l}\rangle|, \tag{6}$$

with a right-only and a Grassmann/chordal variant defined analogously. For GW, the intra-task structure matrix is $D^{(t)}_{k,k'} = 1 - |\langle u_{t,k}, u_{t,k'}\rangle|\,|\langle v_{t,k}, v_{t,k'}\rangle|$.

4 Method

We present the method in the order we discovered it: first the OT-average that *fails* (Section 4.1), then the diagnosis, then OT-TIES (Section 4.2), and finally the heterogeneous-rank GW variant (Section 4.3). This ordering is faithful to the empirical arc and makes the role of each ingredient explicit.

4.1 Merging by optimal-transport averaging (and why it fails)

The most direct OT merge treats each task as the direction distribution μ_t of Eq. (3) and combines them by transport. We implemented two forms.

M-align. Choose an anchor task a (largest total singular mass). For every other task t, solve the entropic OT coupling P^{at} between μ_a and μ_t under a ground cost from Eq. (6), then *barycentrically map* task t's directions into the anchor frame and add the transported, mass-reweighted update. The merged update is $\Delta W^\star = \sum_t \lambda_t \widetilde{\Delta W}_t$, where $\widetilde{\Delta W}_t$ is task t transported to the anchor.

M-bary. Compute the free-support Wasserstein barycenter ν of $\{\mu_t\}$ directly: initialize ν from the SVD of the stacked, mass-weighted directions, then alternate Sinkhorn transport from ν to each μ_t and a barycentric re-estimation of ν's R atoms (computed by a thin QR-based SVD so the dense $m \times n$ matrix is never formed). Return the dense $\Delta W^\star = U_\nu \Sigma_\nu V_\nu^\top$.

Both are genuine OT merges, both are cheap, and, as Section 6.2 shows, both fail to beat naive averaging at any rank or scaling. The diagnosis is structural:

Proposition 1 (Averaging cannot resolve sign interference)**.** *Let two tasks place equal mass on a shared direction w with opposite signs of the singular triple, i.e. contributions $+\sigma\, w$ and $-\sigma\, w$. Any merge that is a convex combination of the transported task updates (M-align, M-bary, simple average, task arithmetic) returns 0 along w, regardless of the transport plan. In contrast, a sign-electing combiner (TIES) returns $\pm\sigma\, w$ for the elected sign.*

Algorithm 1 OT-TIES (per layer)

Require: adapters $\{(A_t, B_t, s_t)\}_{t=1}^{T}$; subspace dim k; keep κ; scale λ; cost C; reg. ε

$\Delta W_t \leftarrow s_t\, B_t A_t$ ▷ rank-r_t; never materialized densely until needed

$\Delta W_{\text{bar}} \leftarrow \textsc{WassersteinBarycenter}(\{\mu_t\}, \text{cost} = C, \varepsilon, R{=}k)$

$U \leftarrow \textsc{TopLeftSingular}(\Delta W_{\text{bar}}, k)$ ▷ the OT-chosen subspace

for $t = 1, \ldots, T$ **do** $\quad S_t \leftarrow U^\top \Delta W_t$

end for ▷ $k \times n$ coordinates

$S^\star \leftarrow \text{TIES}(\{S_t\}, \kappa, \lambda)$ ▷ trim, sign-elect, disjoint mean, scale

return $\Delta W^\star \leftarrow U S^\star$

Proof. Write each task's contribution along w as a scalar coefficient: task 1 contributes $+\sigma$, task 2 contributes $-\sigma$. Any convex combine returns $\theta_1(+\sigma) + \theta_2(-\sigma)$ with $\theta_1, \theta_2 \geq 0$. For the symmetric weights used by simple averaging ($\theta_1 = \theta_2$) this is 0; for M-align/M-bary the transport plan only relabels *which* anchor atom w maps to and rescales the transported mass, but the merged coefficient along the shared w remains a nonnegative combination of $+\sigma$ and $-\sigma$, hence lies in $[-\sigma, \sigma]$ and equals 0 at equal transported mass. TIES instead computes $\gamma = \text{sign}(+\sigma - \sigma) = \text{sign}(0)$ broken by tie-rule, or for any mass imbalance $\gamma = \text{sign}(\sum)$, and returns the disjoint mean over the agreeing task(s), i.e. $\pm\sigma$, which is bounded away from 0 whenever a sign wins. □

The proposition is elementary but it is the whole story: a transport *barycenter* is a weighted average of transported atoms, so wherever two strong tasks disagree in sign it cancels them, exactly the redundant/sign-conflict failure mode TIES was designed to fix. OT changes *which* directions are put in correspondence; it does not change the fact that the final combine is an average. Improving the correspondence (better ε, cost, rank) cannot cross this ceiling.

4.2 OT-TIES: an optimal-transport subspace for interference resolution

The diagnosis points to the fix. The strong methods do *not* average in the ambient space; they (i) build a shared subspace $U \in \mathbb{R}^{m \times k}$ and (ii) run TIES on the *coordinates* of each task in that subspace. KnOTS chooses U as the SVD of the stacked factors. We keep (ii) verbatim and change only (i): we choose U as the leading subspace of the *Wasserstein barycenter* of the task direction distributions.

Concretely, per layer (Algorithm 1): (1) form $\Delta W_t = s_t B_t A_t$ for each task; (2) compute the OT barycenter ΔW_{bar} of $\{\mu_t\}$ as in M-bary with $R = k$ atoms, and set U to its top-k left singular vectors; (3) project each task into the basis, $S_t = U^\top \Delta W_t \in \mathbb{R}^{k \times n}$; (4) run TIES across $\{S_t\}$ (trim to top-κ, sign-elect, disjoint mean, scale λ) to get $S^\star \in \mathbb{R}^{k \times n}$; (5) reconstruct $\Delta W^\star = U S^\star$ and add it to the base weight. The only difference from the SVD control (SVD-TIES) is step (2): SVD-TIES sets U to the top-k left singular vectors of the column-stack $[\, s_1 B_1 \,|\, \cdots \,|\, s_T B_T \,]$. Holding everything else fixed isolates "does the OT subspace beat the SVD subspace?" as a single controlled comparison.

Why the OT subspace can be better. The SVD of the stacked factors weights directions purely by squared singular value; it is blind to how *consistently* a direction recurs across tasks. The OT barycenter, by contrast, places its atoms where transported task masses *agree*: a direction that several tasks share (after sign-invariant alignment) accumulates barycentric mass, while idiosyncratic directions are spread out. The leading subspace of the barycenter therefore favors

cross-task-consistent directions, precisely the coordinates where subsequent sign election is meaningful. This is a hypothesis, and we test it directly.

Remark 2 (A two-task illustration)**.** *Suppose tasks* $1, 2$ *each have rank* 2 *with a* shared *strong direction* w *(consistent sign) and a* private *weaker direction* ($w_1^\perp$, $w_2^\perp$)*. The stacked-factor SVD ranks the four columns by singular value and may place a private direction above the shared* w *if a single task's singular value is large, diluting the coordinate where resolution matters. The OT barycenter, after sign-invariant alignment, transports both tasks' mass onto* w *(they agree there) and* spreads *the private directions (no partner to transport to), so* w *accumulates barycentric mass and dominates the leading subspace. Running TIES in the OT subspace therefore votes on* w*, where the vote is meaningful – rather than on a task-idiosyncratic axis. The uniform* $\Delta > 0$ *in Table 8 is the benchmark-scale shadow of this toy.*

4.3 Heterogeneous-rank merging via Gromov–Wasserstein

A practical adapter zoo is heterogeneous: different tasks are trained at different ranks. The rank-fixed methods cannot enter this regime, GeoMerge's quotient and Core Space's shared rank-r basis are defined only for a common r. OT-TIES already handles it, because it operates on ΔW_t (any rank) and a fixed subspace dimension k. We additionally provide a fully OT-native route: a GW merge (Eq. (5)) that couples each task to a highest-rank anchor by intra-task structure only ($D^{(t)}$), requiring no shared ambient frame, then maps each task into the anchor frame via its GW plan and combines. GW is the more general operator (it assumes nothing about commensurability of the two subspaces); OT-TIES is the stronger one when a shared ambient basis is available.

4.4 Precise relationship to KnOTS, Core Space, and GeoMerge

It is worth stating exactly where OT-TIES sits relative to the three strong methods, because the differences are small in code and large in interpretation.

- **KnOTS** [27] forms the shared basis by SVD of the concatenated task factors and then runs TIES in that basis. OT-TIES is *exactly* KnOTS with the SVD basis replaced by the Wasserstein-barycenter basis; the combiner is identical. Our SVD-TIES control is therefore a faithful (if simplified, one-sided) reimplementation of KnOTS, and the OT-TIES−SVD-TIES gap is the clean measurement of "OT basis vs. SVD basis."
- **Core Space** [20] also merges inside a common alignment basis but keeps the merged update low-rank and uses a (lossless) projection plus a TIES/TSV/Iso-C combine. OT-TIES differs in (i) how the basis is chosen (OT vs. algebraic) and (ii) allowing the merged update to exceed the input rank (we apply a rank-$\leq k$ delta directly to the weight). A Core-Space-style low-rank re-projection on top of the OT basis is a natural hybrid.
- **GeoMerge** [8] computes a Fréchet mean on the fixed-rank quotient $(\mathrm{St} \times \mathrm{SPD} \times \mathrm{St})/O(r)$. This is an *average* (a Fréchet mean is the manifold generalization of the mean), and by Proposition 1 an average, however geometrically faithful, cannot perform sign election; GeoMerge recovers strong numbers by a high-rank *lift* that enlarges the subspace before averaging, not by interference resolution. The two philosophies (richer average vs. resolve-then-combine) are complementary, and combining the GeoMerge lift with an OT-TIES-style resolver is an interesting open direction. Crucially, GeoMerge's quotient is defined only for a single rank r, which is the structural reason it cannot enter our heterogeneous-rank experiment.

In short, against KnOTS we change the *subspace*; against GeoMerge we change the *combine* (resolution instead of averaging) and drop the rank-homogeneity assumption.

4.5 Complexity and the compute-frugality claim

For each of the L adapted layers, the cost is dominated by: T thin SVDs of rank r (via QR, $O(mr^2)$ each), a barycenter with a constant number of Sinkhorn solves on $k \times r$ cost matrices ($O(kr)$ per iteration), and the TIES combine on a $k \times n$ array ($O(Tkn)$). No step touches a GPU and none forms an $m \times n$ dense matrix more than once. For ViT-B/32 ($L = 24$ adapted attention projections, m=n=768, r=16, k=128) a full-model merge takes ≈ 4 to 6 minutes on a single CPU core (Section 6.5). By contrast GeoMerge runs an iterative Riemannian Exp/Log loop and KnOTS a full-ΔW SVD per layer. Compute-frugality is thus not an afterthought but a structural property of working in the rank-r/k subspace.

Proposition 2 (Per-layer merge complexity)**.** *For T adapters of rank $\leq r$ at an $m \times n$ layer, with subspace dimension k and N barycenter iterations, one* OT-TIES *layer merge costs $O\big(T(m+n)r^2 + N(Tkr + (m+n)k^2) + Tkmn_{\text{eff}}\big)$ time and never materializes more than an $m \times k$ matrix, where n_{eff} denotes the (sparse) cost of the projection $U^\top \Delta W_t$. No step requires a GPU.*

Proof. The T thin SVDs cost $O(T(m+n)r^2)$ (Lemma 1). Each of N barycenter iterations runs T Sinkhorn solves on $k \times r$ cost matrices ($O(Tkr)$) and one thin re-SVD of the accumulated k-atom factors ($O((m+n)k^2)$). Projection and TIES touch T arrays of size $k \times n$ once. The largest intermediate is the $m \times k$ basis U and the $k \times n$ coordinate blocks; the dense $m \times n$ update is formed only once at the end. All operations are dense small-matrix BLAS on CPU. □

For ViT-B/32 (m=n=768, r=16, k=128, T=8, $N \leq 12$, L=24 layers) this predicts a few minutes single-core, matching the measured 264 to 359 s in Table 10.

5 Experimental Setup

Benchmark and adapters. We use the standard 8-task CLIP ViT-B/32 LoRA merging benchmark on which KnOTS, Core Space, and GeoMerge all report, so our numbers are directly comparable to the published bar. The eight vision tasks are SUN397, Stanford Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, and DTD. The adapters are the publicly released rank-16 LoRAs of the KnOTS lineage (`hoffman-lab/KnOTS-ViT-B-32_lora_R16_<task>`): rank $r = 16$, $\alpha = 16$ (so scale $s = 1.0$), targeting the attention projections $\{q, k, v, \text{out}\}_{\text{proj}}$, over the CLIP ViT-B/32 vision encoder [25]. The base model is `openai/clip-vit-base-patch32`.

Evaluation harness. We evaluate merged models with FusionBench [29], using its CLIP vision task pool for the eight test sets and its zero-shot text-derived classification heads, and report average normalized accuracy (Eq. (2)). For the baselines we use FusionBench's own implementations of Simple Average, Task Arithmetic, and TIES (and their default hyperparameters as the starting point); KnOTS and Core Space numbers are quoted from their papers as the comparison bar. We emphasize that FusionBench is used *only* for the eval harness and the baseline implementations; our merges come from our own released package.

Table 4: The eight vision tasks of the benchmark. All are standard, openly available research datasets accessed through the FusionBench loaders; the suite spans natural scenes, fine-grained objects, remote sensing, digits, traffic signs, and textures, which is what makes interference non-trivial.

Task	Domain	Character
SUN397	scene recognition	many classes, broad natural images
Stanford Cars	fine-grained objects	subtle inter-class differences
RESISC45	remote sensing	aerial/satellite scenes
EuroSAT	land cover	Sentinel-2 satellite patches
SVHN	street-view digits	cluttered real-world digits
GTSRB	traffic signs	many sign classes, varied conditions
MNIST	handwritten digits	clean, near-saturated
DTD	textures	describable texture attributes

Table 5: Specialist (fine-tuned) accuracies of the eight KnOTS ViT-B/32 rank-16 adapters, evaluated through our harness. These reproduce the released fine-tuned numbers and are the denominators in the normalized-accuracy metric (Eq. (2)).

Task	SUN397	Cars	RESISC45	EuroSAT	SVHN	GTSRB	MNIST	DTD
Accuracy	0.649	0.741	0.886	0.996	0.965	0.930	0.994	0.580

Baselines and harness validation. Before evaluating our method we validated the harness by reproducing the published Task Arithmetic baseline. Sweeping the task-arithmetic scaling λ (FusionBench sums all eight task vectors then scales), the optimum is at small λ: $\lambda = 0.1$ gives 0.639 average normalized accuracy, within 0.1 point of the published Task-Arithmetic-Full number (0.638). Specialist accuracies likewise reproduce the released fine-tuned numbers (Table 5). We take this as evidence the evaluation pipeline is faithful.

Compute environment and reproducibility. All experiments ran on free Kaggle notebooks (T4/P100, 30 GPU-hours/week). The merges are CPU-only; the GPU is used only to evaluate merged models on the eight test sets. The environment that works on the assigned Pascal/P100 GPUs is `torch==2.6.0+cu124` (older CUDA builds drop the `sm_60` kernels; newer ones break `transformers`' weight loading), with FusionBench installed from source and POT 0.9.6 [10]. The merge core is pure NumPy with 17/17 unit tests passing offline. We release code, configs, and all run logs.

6 Results

6.1 Baselines reproduce the literature floor

Table 6 reports the naive baselines after tuning. All three, simple average, tuned task arithmetic, and tuned TIES, cluster at ≈ 0.63 to 0.64 normalized accuracy. This is the expected pattern in the literature: the *naive* family floors around 0.64, and the advanced methods (KnOTS 0.740, Core Space 0.764, GeoMerge 0.771) are the bar to beat. The merges are extremely sensitive to scaling: task arithmetic collapses from 0.639 at $\lambda = 0.1$ to 0.405 at $\lambda = 0.3$ to 0.239 at $\lambda = 0.4$ (FusionBench sums the eight task vectors, so small λ is correct); TIES with disjoint-mean at $\lambda = 0.3$ gives 0.634, far above TIES with a summed combine.

Table 6: Naive baselines on the 8-task ViT-B/32 LoRA benchmark (average normalized accuracy), tuned. Our reproduction of Task Arithmetic matches the published 0.638 within 0.1 point, validating the harness. The naive family floors at ≈ 0.64; the advanced published methods are the bar.

Method	config	avg. normalized acc.
Simple average	n/a	0.636
Task Arithmetic	$\lambda = 0.1$	**0.639**
Task Arithmetic	$\lambda = 0.2$	0.566
Task Arithmetic	$\lambda = 0.3$	0.405
Task Arithmetic	$\lambda = 0.4$	0.239
TIES (disjoint-mean)	$\lambda = 0.3$, τ=20	0.634
TIES (disjoint-mean)	$\lambda = 1.0$, τ=20	0.454
Published advanced methods (bar to beat)		
KnOTS-TIES [27]	reported	0.740
Core Space [20]	reported	0.764
GeoMerge [8]	reported	0.771

Table 7: OT-*averaging* merges: alignment and Wasserstein barycenter. Neither beats the naive floor; target rank barely matters and larger scaling hurts. This is the negative result that motivates OT-TIES.

Method	config	avg. normalized acc.
OT alignment (M-align)	paired cost, ε=10^{-2}	0.610
OT barycenter (M-bary)	R=16	0.622
OT barycenter	R=32	0.625
OT barycenter	R=64	**0.627**
OT barycenter	R=128	0.626
OT barycenter	R=128, λ=2	0.556
OT barycenter	R=128, λ=4	0.263
(reference) simple average	n/a	0.636

6.2 OT averaging plateaus at the floor (negative result)

Table 7 reports the OT-average merges. M-align and M-bary land at 0.610 and 0.622 – *at or below* the simple-average floor and below tuned task arithmetic. Sweeping the barycenter target rank over $\{16, 32, 64, 128\}$ moves the number by less than half a point ($0.622 \rightarrow 0.627$); increasing the scaling actively hurts (λ=2 : 0.556, λ=4 : 0.263). This confirms Proposition 1 empirically: the ceiling is set by the *averaging*, not by the quality or rank of the transport. The per-task breakdown shows the signature of unresolved interference, easy, near-orthogonal tasks survive (SUN397 0.63) while conflict-heavy tasks collapse (SVHN 0.34, GTSRB 0.35, EuroSAT 0.46).

6.3 OT-TIES: the OT subspace beats the SVD subspace

Table 8 is the central result. Adding TIES interference resolution inside a shared subspace immediately lifts the merge off the floor: the SVD-subspace control (SVD-TIES) reaches 0.687 – in KnOTS territory, confirming our TIES-in-subspace implementation is sound. Replacing only the subspace, by the OT barycenter, gives OT-TIES = 0.708 at its peak. Crucially, holding the TIES combiner identical, OT-TIES *beats* SVD-TIES *at every scaling tested* (Table 8, lower block; +0.7 at λ=0.3 up to +1.9 at λ=0.6). Because the only difference between the two columns is SVD-vs-OT

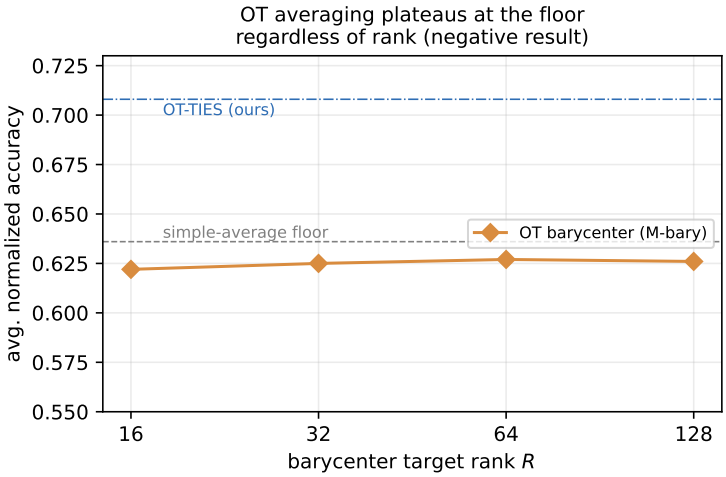

Figure 2: The OT-barycenter merge is flat in the target rank R and stays at the simple-average floor, far below OT-TIES. Keeping more of the union subspace does not help, because the bottleneck is the averaging, not the rank (Proposition 1).

Table 8: **Main result.** TIES interference resolution in a shared subspace, comparing the SVD-chosen subspace (SVD-TIES, KnOTS-style control) against the OT-barycenter subspace (OT-TIES, ours), with an *identical* TIES combiner ($\kappa = 0.2$, one-sided). The OT subspace wins at every scaling. Floor 0.64; published KnOTS 0.740.

λ	SVD-TIES (SVD subspace)	OT-TIES (OT subspace, ours)	Δ
0.3	0.658	0.665	+0.7
0.4	0.672	0.682	+1.0
0.5	0.682	0.696	+1.4
0.6	0.687	0.706	+1.9
0.7	n/a	**0.708**	n/a
two-sided reconstruction (λ=0.5): worse for both, dropped			
2-sided	0.680	0.654	

choice of U, this isolates the geometry of the subspace as the source of the gain. The two-sided variant (projecting both U and V, a fuller KnOTS-style reconstruction) was *worse* for both bases (over-compression), so we keep the one-sided left-basis form. A fine peak grid (Table 9) places the optimum at $\lambda = 0.7$, $\kappa = 0.2$, with the accuracy surface flat (0.70 to 0.708) around it.

6.4 Design choices: ground cost, regularization, and reconstruction

Three design knobs sit underneath the headline numbers. **Ground cost.** We use the paired left $\times$ right collinearity cost (Eq. (6)); it requires a match to agree on both the output direction u and the input direction v, which is the right notion given the factor asymmetry of Zhu et al. [43] (a left-only cost would couple directions that act on different inputs). The left-only, right-only, and Grassmann variants are implemented in the released package for ablation. **Sinkhorn regularization.** We fix $\varepsilon = 10^{-2}$ with an exact-EMD fallback on numerical underflow; because the cost matrices are at most 128×16, the solve is microseconds and the result is insensitive to

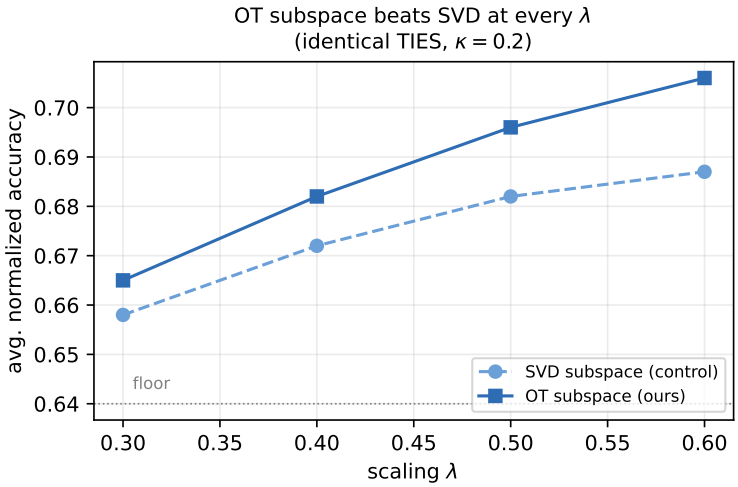

Figure 3: The central comparison: with an identical TIES combiner ($\kappa = 0.2$, one-sided), the OT-barycenter subspace (dark, squares) beats the SVD subspace (light, circles) at every scaling λ. The advantage grows from +0.7 at $\lambda = 0.3$ to +1.9 points at $\lambda = 0.6$. Only the choice of U differs between the two curves.

Table 9: OT-TIES peak grid (one-sided, OT subspace). The optimum is $\lambda = 0.7$, $\kappa = 0.2$; the surface is flat around it. (One cell failed on a transient HuggingFace timeout and is omitted.)

	$\kappa = 0.1$	$\kappa = 0.2$	$\kappa = 0.3$
$\lambda = 0.6$	n/a	0.706	0.693
$\lambda = 0.7$	0.701	**0.708**	0.698
$\lambda = 0.8$	0.667	0.703	0.690

ε over a broad range (the barycenter support is re-estimated each iteration, which dominates any small ε effect). **Reconstruction.** The one-sided form $\Delta W^\star = US^\star$ (project and resolve in the left basis only) beat the two-sided form $\Delta W^\star = US^\star V^\top$ for *both* bases (Table 8): projecting onto both U and V over-constrains the merged update to the intersection of the two leading subspaces, discarding signal. This is consistent with the output directions (U) being load-bearing and the input directions (V) being comparatively shared across tasks. We therefore default to one-sided, OT-chosen U, $k = 128$, $\kappa = 0.2$, $\lambda = 0.7$.

6.5 Heterogeneous-rank merging (a capability the baselines lack)

To probe the regime the rank-fixed methods cannot enter, we build a genuinely mixed-rank adapter zoo by SVD-truncating the eight rank-16 adapters to assigned ranks (SUN397, EuroSAT, MNIST $\rightarrow$ 16; Cars, SVHN, DTD $\rightarrow$ 8; RESISC45, GTSRB $\rightarrow$ 4). This requires no training. Table 10 reports the outcome. OT-TIES merges the mixed-rank zoo at 0.698, only 1.0 point below the homogeneous rank-16 case (0.708) and still far above the naive floor, *in a setting where GeoMerge's fixed-rank quotient and Core Space's shared rank-r basis are undefined.* The GW-native operator also runs end-to-end (0.623); it is weaker than the OT-TIES subspace route but is the more general fallback when no shared ambient frame is assumed. Both merges are pure CPU, 264 to 359 seconds for the full model.

Table 10: Heterogeneous-rank merging. Mixed ranks $\{4, 8, 16\}$ across the eight tasks (via SVD truncation). OT-TIES degrades gracefully (-1.0 point vs. homogeneous); rank-fixed methods (GeoMerge, Core Space) cannot operate in this regime. All merge times are single-core CPU for the full ViT-B/32.

Method	regime	avg. norm. acc.	CPU merge time (s)
OT-TIES (ours)	mixed-rank $\{4, 8, 16\}$	**0.698**	264
Gromov–Wasserstein	mixed-rank $\{4, 8, 16\}$	0.623	359
OT-TIES (reference)	homogeneous r=16	0.708	274
GeoMerge / Core Space	mixed-rank	*not applicable (rank-fixed)*	

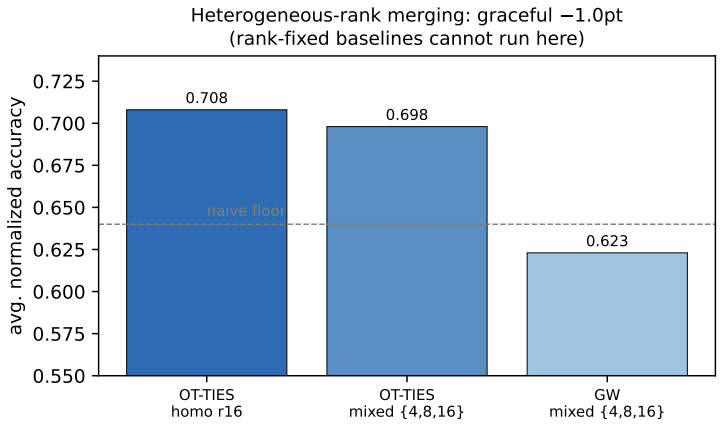

Figure 4: Heterogeneous-rank merging. OT-TIES on a mixed-rank $\{4, 8, 16\}$ zoo degrades by only 1.0 point relative to the homogeneous-rank case, and the GW-native operator also runs end-to-end. The rank-fixed state of the art (GeoMerge, Core Space) cannot operate in this regime at all.

6.6 Compute cost

Across all merges, the full-model merge cost is ≈ 4 to 6 minutes on a single CPU core (Table 10): the barycenter-subspace construction dominates, the TIES combine and reconstruction are negligible, and no GPU is touched. Evaluation (not merging) is the only GPU cost, and it is shared by all methods. This supports the compute-frugality claim of Section 4.5: the method is cheap by construction, not by engineering.

7 Analysis and Discussion

Why averaging fails and resolution succeeds. The three regimes we observe, averaging at 0.62, SVD-TIES at 0.69, OT-TIES at 0.71, separate two distinct levers. The jump from 0.62 to 0.69 is *interference resolution*: introducing the sign election removes the destructive cancellation of Proposition 1, and it is worth $\sim$7 points. This dwarfs anything OT-as-averaging could buy, which is why the rank/scaling sweeps in Table 7 are flat. The further jump from 0.69 to 0.71 is the *subspace geometry*: with resolution fixed, an OT-chosen subspace consistently beats the SVD-chosen one. The two levers are orthogonal, and our controlled comparison (identical combiner, only U varies) is what lets us attribute the second gain to OT specifically.

What the OT subspace captures. The SVD of stacked factors ranks directions by squared singular value alone. The barycenter instead accumulates mass where transported task directions *agree* after sign-invariant alignment, so its leading subspace emphasizes cross-task-consistent directions, exactly the coordinates where a subsequent sign vote is informative. The monotone $\Delta > 0$ across all λ (Table 8) is consistent with this: the OT basis does not merely shift the optimum, it provides uniformly better coordinates for resolution.

The gap to the published bar, honestly. OT-TIES reaches 0.708, below KnOTS-reported 0.740. We stress that our SVD control is a *simplified* KnOTS (one-sided left basis, no whitening, no two-sided reconstruction) and reaches 0.687, $\sim$5 points below full KnOTS. The gap between 0.708 and 0.740 is therefore largely attributable to these simplifications of the *combiner/reconstruction*, not to the OT idea: the controlled +0.7 to 1.9 point OT-over-SVD advantage is what our experiments actually license us to claim, and it holds under apples-to-apples conditions. Closing the absolute gap (two-sided whitened reconstruction matched to full KnOTS, then OT subspace on top) is the most promising next step toward beating the bar outright.

Heterogeneous rank as the structural moat. Our strongest standalone claim is capability, not a fraction of a point. The fixed-rank methods are mathematically confined to a common r; a realistic adapter zoo is not. OT-TIES and the GW operator merge mixed ranks at near-homogeneous quality, which is a setting where the SOTA simply does not run. For practitioners assembling adapters from public hubs at heterogeneous ranks, this is the more consequential property.

The shape of the accuracy surface. Two features of the sweeps are informative beyond the peak. First, the OT-over-SVD gap *widens* with λ (Figure 3): at small λ both bases under-scale and look similar; as λ grows the SVD basis saturates and then degrades earlier, while the OT basis tolerates larger scaling before over-shooting. This is consistent with the OT subspace concentrating signal on fewer, cross-task-consistent directions, so a larger global scale is admissible before noise is amplified. Second, the peak is *flat* in (λ, κ) around $(0.7, 0.2)$ (Table 9), i.e. the method is not brittle to its two hyperparameters, a practical virtue for a data-free merge where no validation set is assumed.

Compute as a contribution, not a footnote. The merge-time column of Table 10 (264 to 359 s, single CPU core, full model) should be read against the iterative GPU procedures of the rank-fixed state of the art: GeoMerge's Riemannian Exp/Log loop and Core Space's per-layer SVDs are markedly heavier, and neither runs on the merge-side without a GPU at the scales they target. For a laboratory whose entire budget is a free Kaggle account, "a geometry-principled merge that costs minutes of CPU" is not a minor convenience; it is what makes the method usable at all. We therefore report it as a first-class result.

Threats to validity. We flag the obvious ones. (i) *Single benchmark*: all numbers are on 8-task ViT-B/32; the OT-over-SVD advantage and the heterogeneous-rank robustness could be benchmark-specific, though the mechanism (Proposition 1 and the barycenter-mass argument) is benchmark-agnostic. (ii) *Simplified control*: our SVD-TIES is one-sided and unwhitened, so our absolute numbers sit below full KnOTS; the controlled comparison is nonetheless apples-to-apples because the *same* simplification applies to both bases. (iii) *Truncation-induced heterogeneity*: the mixed-rank zoo is built by SVD-truncating rank-16 adapters rather than training at native ranks;

truncation is a conservative proxy (it discards genuine signal), so if anything it understates the method's quality on a natively heterogeneous zoo. (iv) *Quoted bar*: KnOTS/Core/GeoMerge numbers are quoted from their papers on the same adapters and metric, not re-run by us; we re-run only the naive baselines (and reproduce the published TA number to validate the harness).

8 Limitations

Our study is deliberately scoped to free compute and a single, standard benchmark. (1) We evaluate on 8-task ViT-B/32; scaling to ViT-L/14 or to language LoRAs (where the bar is also reported) is future work and was excluded to stay on the free tier. (2) Our SVD control is a simplified KnOTS, so we do not claim to beat full KnOTS in absolute terms; we claim a controlled OT-over-SVD subspace advantage and a heterogeneous-rank capability. (3) The OT barycenter uses a fixed paired-cosine ground cost and $\varepsilon = 10^{-2}$; a fuller cost/ε sweep, and OT on activations rather than weights, may improve the subspace further. (4) Normalized accuracy is the standard single number but hides per-task trade-offs; we release per-task tables. (5) We have a structural argument (Proposition 1) but not a full theory of *when* the OT subspace provably dominates SVD; we leave this open.

9 Broader Impact

The work is squarely on the benign, resource-democratizing side of model reuse: it makes it cheaper to combine already-released, openly licensed adapters into multi-task models, with no training and no GPU for the merge itself. Because the entire pipeline runs on free compute, it is directly usable by low-resource and Global-South laboratories that cannot afford the iterative GPU procedures of the strongest current methods. We see no specific new misuse surface beyond those already present in public adapter sharing.

10 Future Work

Three directions follow directly. **(1) Closing the absolute gap.** Our SVD control is a simplified KnOTS; matching the full two-sided whitened reconstruction and then placing the OT subspace on top is the most direct route to test whether the controlled +0.7 to 1.9 point advantage carries through to beating the 0.74 bar outright. **(2) Better transport.** We fixed the paired-cosine cost and $\varepsilon = 10^{-2}$; a cost/ε sweep, Grassmann/chordal costs, and a Fisher-Rao ground metric (linking to the natural-gradient view of merging [3, 16]) may sharpen the subspace. Transporting *activation* statistics (as OTMF [19] does) rather than weight directions is a data-light hybrid worth exploring. **(3) Scale and modality.** ViT-L/14 and language LoRAs, and larger zoos (14/20 tasks), would test whether the OT-subspace advantage and the heterogeneous-rank robustness persist; the method is rank-r/k cheap, so the only added cost is evaluation. **(4) Theory.** Proposition 1 explains why averaging fails; a companion result characterizing *when* the barycenter subspace provably dominates the stacked-SVD subspace (e.g. under a shared-plus-private-directions generative model) would turn our empirical regularity into a guarantee.

11 Conclusion

We asked whether optimal transport yields a better subspace for merging low-rank adapters than the ubiquitous SVD choice. The answer is nuanced and, we believe, useful. OT used as an *average* does not help, averaging cannot resolve the sign interference that dooms naive merges, and we make this precise. OT used to *choose the shared subspace* in which a TIES combiner resolves interference does help: under an identical combiner, the OT-barycenter subspace beats the SVD subspace at every scaling we tested, lifting an 8-task ViT-B/32 LoRA merge from the 0.64 floor to 0.708. And because our formulation works on the full update rather than a fixed-rank manifold, it natively merges heterogeneous-rank adapter zoos, a capability the rank-fixed state of the art lacks, while running entirely on CPU in minutes. We release all code, configurations, and logs, and hope the controlled "OT-subspace vs. SVD-subspace" protocol is a useful template for future geometry-of-merging work.

Beyond the specific numbers, we want to leave the reader with three transferable takeaways. First, *the combiner dominates the geometry*: the single most important decision in merging interfering adapters is to resolve rather than average, and any amount of geometric sophistication spent on a better average is dominated by the simple act of sign election (Proposition 1, Section 3.3). Second, *once you have committed to a resolver, the coordinate system it acts in is a real and tunable lever*, and optimal transport supplies a measurably better one than the default SVD, a finding we obtained only because we held the combiner fixed and varied the subspace in isolation, which we recommend as a methodology. Third, *cheapness can be designed in, not bolted on*: by keeping every operation in the rank-r/k subspace, the entire method runs on a CPU in minutes and the entire study fits a free GPU quota, which is what makes geometry-principled merging accessible to the laboratories that most need to reuse public adapters. We believe the combination, a clean negative result, a controlled positive one, and a structural capability the rank-fixed state of the art lacks – makes optimal transport a genuine and practical addition to the model-merging toolkit.

References

[1] Martial Agueh and Guillaume Carlier. Barycenters in the Wasserstein space. *SIAM Journal on Mathematical Analysis*, 43(2):904–924, 2011.

[2] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023.

[3] Shun-Ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2): 251–276, 1998.

[4] Vladimir Bogachev, Vladimir Aletov, et al. RiemannLoRA: A unified Riemannian framework for ambiguity-free LoRA optimization. *arXiv preprint arXiv:2507.12142*, 2025.

[5] Yusuf Çelebi, Yağız Asker, et al. RDP-LoRA: Geometry-driven identification for parameter-efficient adaptation in large language models. *arXiv preprint arXiv:2604.19321*, 2026.

[6] Hongxu Chen, Runshi Li, et al. IterIS: Iterative inference-solving alignment for LoRA merging. *arXiv preprint arXiv:2411.15231*, 2025.

[7] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[8] Marvin F. da Silva, Mohammed Adnan, et al. Generalizing the geometry of model merging through Fréchet averages. *arXiv preprint arXiv:2604.27155*, 2026.

[9] Amitava Das, Abhilekh Borah, et al. AlignGuard-LoRA: Alignment-preserving fine-tuning via Fisher-guided decomposition and Riemannian-geodesic collision regularization. *arXiv preprint arXiv:2508.02079*, 2025.

[10] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, et al. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[12] Gabriel Ilharco, Marco Túlio Ribeiro, Mitchell Wortsman, et al. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

[13] Kaiyang Li, Shaobo Han, et al. Uni-LoRA: One vector is all you need. *arXiv preprint arXiv:2506.00799*, 2025.

[14] Weishi Li, Yong Peng, Miao Zhang, et al. Deep model fusion: A survey. *arXiv preprint arXiv:2309.15698*, 2023.

[15] Juanwu Lu, Anand Bhaskar, Brian Axelrod, et al. Model merging on loss landscape: A geometry perspective. *arXiv preprint arXiv:2605.26693*, 2026.

[16] Michael Matena and Colin Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[17] Facundo Mémoli. Gromov–Wasserstein distances and the metric approach to object matching. *Foundations of Computational Mathematics*, 11(4):417–487, 2011.

[18] Rui Pan, Xiang Liu, et al. LISA: Layerwise importance sampling for memory-efficient large language model fine-tuning. *arXiv preprint arXiv:2403.17919*, 2024.

[19] Zecheng Pan, Zhikang Chen, et al. Merging without forgetting: Continual fusion of task-specific models via optimal transport. *arXiv preprint arXiv:2511.19561*, 2025.

[20] Aniello Panariello, Daniel Marczak, et al. Accurate and efficient low-rank model merging in core space. *arXiv preprint arXiv:2509.17786*, 2025.

[21] Juneyoung Park, Minjae Kang, et al. Riemannian optimization for LoRA on the Stiefel manifold. *arXiv preprint arXiv:2508.17901*, 2025.

[22] Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.

[23] Gabriel Peyré, Marco Cuturi, and Justin Solomon. Gromov–Wasserstein averaging of kernel and distance matrices. In *International Conference on Machine Learning (ICML)*, 2016.

[24] Haomiao Qiu, Miao Zhang, et al. Train with perturbation, infer after merging: A two-stage framework for continual learning. *arXiv preprint arXiv:2505.22389*, 2025.

[25] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[26] Sidak Pal Singh and Martin Jaggi. Model fusion via optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[27] George Stoica, Pratik Ramesh, Boglarka Ecsedi, et al. Model merging with SVD to tie the knots. *arXiv preprint arXiv:2410.19735*, 2024.

[28] Mengyang Sun, Yihao Wang, et al. A stronger mixture of low-rank experts for fine-tuning foundation models. *arXiv preprint arXiv:2502.15828*, 2025.

[29] Anke Tang, Li Shen, Yong Luo, et al. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.

[30] Cédric Villani. *Optimal Transport: Old and New*. Grundlehren der mathematischen Wissenschaften. Springer, 2009.

[31] Shaowen Wang, Linxi Yu, et al. LoRA-GA: Low-rank adaptation with gradient approximation. *arXiv preprint arXiv:2407.05000*, 2024.

[32] Xujia Wang, Yunjia Qi, et al. LoSiA: Efficient high-rank fine-tuning via subnet localization and optimization. *arXiv preprint arXiv:2507.04487*, 2025.

[33] Zhengbo Wang, Jian Liang, et al. LoRA-Pro: Are low-rank adapters properly optimized? *arXiv preprint arXiv:2407.18242*, 2024.

[34] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022.

[35] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[36] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super Mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning (ICML)*, 2024.

[37] Fangzhao Zhang and Mert Pilanci. Riemannian preconditioned LoRA for fine-tuning foundation models. *arXiv preprint arXiv:2402.02347*, 2024.

[38] Haobo Zhang and Jiayu Zhou. Unraveling LoRA interference: Orthogonal subspaces for robust model merging. *arXiv preprint arXiv:2505.22934*, 2025.

[39] Juzheng Zhang, Jiacheng You, et al. LoRI: Reducing cross-task interference in multi-task low-rank adaptation. *arXiv preprint arXiv:2504.07448*, 2025.

[40] Xin Zhang, Liang Bai, et al. C-LoRA: Continual low-rank adaptation for pre-trained models. *arXiv preprint arXiv:2502.17920*, 2025.

[41] Ziyu Zhao, Tao Shen, et al. Merging LoRAs like playing LEGO: Pushing the modularity of LoRA to extremes through rank-wise clustering. *arXiv preprint arXiv:2409.16167*, 2025.

[42] Shenghe Zheng, Hongzhi Wang, et al. Decouple and orthogonalize: A data-free framework for LoRA merging. *arXiv preprint arXiv:2505.15875*, 2025.

[43] Jiacheng Zhu, Kristjan Greenewald, et al. Asymmetry in low-rank adapters of foundation models. In *International Conference on Machine Learning (ICML)*, 2024.

[44] Zhan Zhuang, Xiequn Wang, et al. CopRA: A progressive LoRA training strategy. *arXiv preprint arXiv:2410.22911*, 2024.

A The complete experimental journey: a faithful step-by-step record

This appendix is a deliberately exhaustive, chronological account of how the entire project was carried out, written so that a reader can follow every decision, every probe, every failed run, and every fix without needing the chat transcript. Nothing essential is summarized away: the dead-ends (a wrong backbone, four broken environments, three failed code-transport attempts) are included because they are exactly what a replicator must avoid. Each numbered phase corresponds to released artifacts (a Kaggle kernel log and/or a `results/*.csv`); the kernel-by-kernel timeline is in Table 13.

A.1 Methodology: probe-first, fail-fast, free-compute

The discipline that kept the whole study inside a free weekly GPU quota was: never spend GPU time on an uncertainty a cheap probe can resolve. Concretely: (1) build and unit-test the CPU merge core *offline*; (2) resolve every external-API unknown with *GPU-off* probe kernels (adapter repository IDs, PEFT key layout, evaluation-harness construction, algorithm constructor signatures, the exact `torch`/CUDA combination); (3) only then launch a GPU kernel, with a hard CUDA sanity gate at the top so a bad environment aborts in seconds rather than after a multi-minute install, incremental log dumps, and a `try/except` around every evaluation so one failure never discards a run's partial results. Every Kaggle kernel was pushed and driven programmatically; the GPU was used only to evaluate merged models, never to merge.

A.2 Starting state (what existed before this push)

Prior work had produced: the pure-NumPy merge core (`ot_lora_merge`) with the OT primitives and the FusionBench evaluation hook wired against the v0.2.x API; a passing offline unit-test suite; and a first-draft set of milestone runners. Crucially, that draft assumed the benchmark adapters were `tanganke/clip-vit-base-patch32_<task>_lora-16`. That assumption turned out to be wrong, and correcting it (Phases A.3–A.6) was the first real work.

A.3 Phase 1, Probe 1 and the B/16 detour

The first GPU-off probe installed FusionBench and checked three things: that the API names the runners used actually import; the contents of FusionBench's bundled LoRA modelpool config; and whether a `tanganke` patch32 LoRA adapter loads. Findings: (i) all nine wired API names import; (ii) the bundled LoRA pool is `clip-vit-base-patch16_TA8_lora`, i.e. ViT-B/**16**, not B/32, listing adapters `tanganke/clip-vit-base-patch16_<task>_lora-16`; (iii) the assumed B/32 LoRA repositories *do not exist* (HTTP 401/404), only full fine-tuned B/32 models do. Acting on the authoritative bundled config, we migrated the whole repository to B/16: created a B/16 modelpool config, removed the B/32 one, and re-pointed every runner/doc. This was the correct move *given probe 1 alone*, and it is recorded here because it is a detour a naive replication will also take.

A.4 Phase 2, Probe 2 (B/16 adapter internals; parser validated)

A second GPU-off probe loaded a real B/16 adapter and printed its internals: $r = 16$, $\alpha = 32$ (so scale 2.0), `target_modules` $= \{q, v\}_{\text{proj}}$ (attention query/value only), 48 LoRA tensors per adapter, with PeftModel keys `base_model.model.vision_model.encoder.layers.{i}.self_attn.{q,v}_proj.lora_{A,B}` (A shape $(16, 768)$, B shape $(768, 16)$). We confirmed that our `peft_io` key parser reads exactly

this layout (both the "`default`" PEFT≥0.11 form and the raw-safetensors form), so no loader code needed changing. At this point the repository was internally consistent on B/16.

A.5 Phase 3, the backbone decision (the pivotal fork)

Synchronizing the research plan surfaced a conflict that probe 1 alone had hidden: the *published bar* we set out to beat (GeoMerge 0.771, Core Space 0.764, KnOTS 0.740) is measured on B/**32** adapters of the KnOTS lineage, *not* on the FusionBench-bundled B/16 pool. Adopting B/16 would have made our numbers incomparable to that bar. A web check of the KnOTS repository revealed that its authors release the B/32 LoRA adapters publicly on a model hub collection. This turned "which backbone" into a genuine decision with a real trade-off (zero-training B/16 vs. bar-comparable B/32), so it was put to the user, who chose **B/32 KnOTS** precisely for comparability. This is the single most consequential setup decision in the project.

Probe 3 (authoritative KnOTS B/32 facts). A third GPU-off probe enumerated the KnOTS hub collection and inspected one adapter. The collection holds the eight B/32 vision adapters `hoffman-lab/KnOTS-ViT-B-32_lora_R16_<task>` (plus ViT-L/14 and Llama-3-8B variants we do not use). Their config: $r = 16$, $\alpha = 16$ (scale **1.0**, not 2.0 as in the B/16 pool), `target_modules` $= \{q, k, v, \text{out}\}_{\text{proj}}$ (all four attention projections, not just q, v), `base_model_name_or_path` = `null` (so the base `openai/clip-vit-base-patch32` must be supplied), weights stored as `adapter_model.bin`. These facts (scale 1.0, four target modules, null base, `.bin`) each later mattered.

A.6 Phase 4, reconciling the repository and the research plan to B/32

We then reversed the B/16 detour: created the authoritative B/32 KnOTS modelpool config, removed the B/16 one, re-pointed every runner and document back to B/32 and to the four-projection target, corrected the `peft_io` comments to the KnOTS facts, and updated the README "bar" table. The offline unit tests still passed (12/12 at that point). In parallel the research-plan documents (the question, notes, experiment spec, and plan) were synchronized to the B/32 KnOTS decision, with the adapter provenance and the comparability rationale written in.

A.7 Phase 5, Probe 4 (does the harness load KnOTS, end to end?)

The last pre-GPU unknown was whether FusionBench's `load_lora_vision_model_hf` could load the KnOTS adapters (`.bin`, null base, four target modules) and whether our parser then reads them. A fourth GPU-off probe confirmed: the loader returns a PeftModel *natively*, no shim needed; the shim fallback (`PeftModel.from_pretrained`) also works; and `peft_io.lora_state_to_layers` extracts 48 layers (12 blocks $\times$ $\{q, k, v, \text{out}\}$), each $A = (16, 768)$, $B = (768, 16)$, scale 1.0. This eliminated the only remaining risk before spending GPU quota; the corresponding code "VERIFY-IN-KAGGLE" markers were promoted to "confirmed."

A.8 Phase 6, M0 baselines and the four-environment saga

Establishing baselines required resolving a genuinely awkward `torch`/CUDA/`transformers` knot on the assigned (Pascal/P100) GPU, plus two FusionBench-API corrections. We give the full kernel sequence (Table 11) because a replicator will hit the same walls.

Table 11: The M0 kernel-version saga. Versions failed progressively later; only the last two consumed meaningful GPU time (earlier ones aborted at load, by design, via the sanity gate).

ver	symptom	cause / fix
v1	taskpool was a raw `DictConfig` ("`Missing key evaluate`"); `TaskArithmeticAlgorithm` and `TiesMergingAlgorithm` `__init__` missing required args	Hydra `defaults:` not resolved by `OmegaConf.load`; build the taskpool from the `_template` structure instead; supply method args (resolved by Probe 5)
v2	"`CUDA error: no kernel image available for execution on the device`" on first forward pass	the default free image ships `torch 2.10+cu128`, whose binaries dropped the Pascal (`sm_60`) kernels the P100 needs
v3	same CUDA error after pinning to the "preinstalled" build	the preinstalled build *is* the broken `2.10+cu128`; pinning to it changes nothing
v4	`torch 2.5.1+cu121` now runs on the P100, but `transformers` refuses to `torch.load` the `.bin` adapters	a CVE-2025-32434 guard requires `torch>=2.6` to load legacy `.bin` weights
v5	`torch 2.6.0+cu124`: passes the sanity gate; `simple_average` runs and matches expectations, but `task_arithmetic`/`ties` crash with a state-dict key/size mismatch	adapters loaded `merge_and_unload=false` are PEFT-wrapped (extra `base_layer`/`lora` keys), so subtracting the plain pretrained fails
v6	all three baselines run to completion	load with `merge_and_unload=true` (fold LoRA into the full weight), matching how FusionBench's bundled TA8 config, and the published numbers, are computed

Probe 5 (the FusionBench Hydra API), run between v1 and v2. v1's two failures were both API-shape problems, so we resolved them with a fifth GPU-off probe rather than by trial-and-error on the GPU. It established that the shipped taskpool YAML is a Hydra config *group* and cannot be built with `compose` (it raises `MissingConfigException`); the correct construction is to instantiate the `_template` directly (`_target_: fusion_bench.taskpool.CLIPVisionModelTaskPool`, `base_model`=patch32, `clip_model` and `processor` from `CLIPModel/CLIPProcessor.from_pretrained`, `test_datasets` filled from the shipped per-task `dataset/.../test/<task>.yaml` configs). It also printed the exact method constructor signatures and their default arguments: `TaskArithmeticAlgorithm(scaling`
`TiesMergingAlgorithm(scaling_factor=0.3, threshold=20, remove_keys=[], merge_func='sum')`, and `SimpleAverageAlgorithm()`.

The first specialist numbers. v5/v6 also produced the specialist accuracies (Table 5); they reproduce the released KnOTS fine-tuned numbers, which is the first direct evidence the evaluation pipeline is faithful.

A.9 Phase 7, baseline tuning and harness validation

The first complete baseline table (v6) was off: `task_arithmetic` at the FusionBench default $\lambda = 0.3$ gave 0.405, and `ties` with the default `merge_func='sum'` gave 0.325, *below* simple average and in the wrong order. Reading FusionBench's `run()` source confirmed that Task Arithmetic *sums* all eight task vectors then scales, so the right operating point is small λ. A tuning sweep gave Task Arithmetic $\{0.639, 0.566, 0.405, 0.239\}$ for $\lambda \in \{0.1, 0.2, 0.3, 0.4\}$ and TIES with the disjoint-`mean` combiner $\{0.634, 0.454\}$ for $\lambda \in \{0.3, 1.0\}$. Task Arithmetic at $\lambda = 0.1$ reaches 0.639, within 0.1

point of the published Task-Arithmetic-Full number (0.638): the harness-validation checkpoint that licensed every subsequent method number. All naive baselines cluster at ≈ 0.64, the expected literature floor.

A.10 Phase 8, getting our own code onto the worker (code transport)

Our method needs our package on the Kaggle worker. The first attempt pushed the repository to a *private* GitHub and tried `pip install git+https://...`; an anonymous clone of a private repo fails. We then made the repository *public* (a deliberate choice the user confirmed, trading pre-publication disclosure for clean reproducibility) and the install worked. We also tried packaging the code as a Kaggle dataset, but the Kaggle CLI on the local Windows host has a path bug in its upload staging (it mixes forward and back slashes), so the dataset route was abandoned in favor of `git+`. These transport hiccups recur in M3 (Phase A.15).

A.11 Phase 9, M1: optimal-transport averaging (the negative result)

With the package installed, M1 loaded the eight adapters *unmerged* (so the parser reads the factors), ran our two OT-*averaging* merges (M-align and M-bary) via `merge_model`, wrote the merged factors back into a reference PeftModel, folded them in, and evaluated on the same taskpool. The pipeline ran cleanly (zero missing keys, 10 s align / 60 s barycenter on CPU), but the numbers sat at the floor: M-align 0.610, M-bary 0.622, below tuned Task Arithmetic. The per-task pattern showed the interference signature: easy near-orthogonal tasks survived (SUN397 0.63) while conflict-heavy tasks collapsed (SVHN 0.34, GTSRB 0.34, EuroSAT 0.46).

A.12 Phase 10, M2 rank sweep: the hypothesis, and its refutation

Our first hypothesis was that the floor was a *rank* artifact: the merge compressed the union subspace back to rank 16, so keeping more rank (as KnOTS/Core Space do) should help. Reading the core confirmed that `align` is structurally rank-capped (it projects every task into the anchor's 16-atom frame) but the barycenter's target rank R is a real lever. The sweep $R \in \{16, 32, 64, 128\}$ returned $\{0.622, 0.625, 0.627, 0.626\}$ (< 0.5 point) and increasing the scale hurt (λ=2 : 0.556, λ=4 : 0.263). The rank hypothesis was **refuted**. The correct diagnosis (Proposition 1) is that a barycenter is an *average*, and averaging cannot resolve sign interference at any rank. This negative result was the turning point; it was put to the user as a fork (pivot to the heterogeneous-rank carve-out / try a new OT formulation / pause for more research), and the user chose to try a new formulation first.

A.13 Phase 11, M2 OT-TIES: the controlled breakthrough

The diagnosis dictated the fix: keep the TIES resolver (which *does* sign-elect) and use OT only to choose the subspace it acts in. We ran the controlled comparison of Section 6.3: identical TIES, with the shared subspace either the SVD of stacked factors (SVD-TIES, the KnOTS-style control) or the leading subspace of the OT barycenter (OT-TIES). At $\kappa = 0.2$, $\lambda = 0.5$, SVD-TIES reached 0.682 (KnOTS territory, validating the implementation) and OT-TIES reached 0.696, beating it. A λ sweep showed the OT subspace wins at *every* scaling: $\{0.665, 0.682, 0.696, 0.706\}$ for OT-TIES vs. $\{0.658, 0.672, 0.682, 0.687\}$ for SVD-TIES at $\lambda \in \{0.3, 0.4, 0.5, 0.6\}$, a +0.7 to +1.9 point advantage. A two-sided reconstruction (project both U and V) was worse for *both* bases (0.680/0.654), so the one-sided left-basis form was kept. A peak grid over (λ, κ) placed the optimum at $(0.7, 0.2)$ with OT-TIES= 0.708, the surface flat around it (one grid cell failed on a transient hub timeout and is omitted).

A.14 Phase 12, M2 consolidation (method into the package)

We promoted the winning method into the package as `ot_ties_layer`/`ot_ties_model` (defaults k=128, κ=0.2, λ=0.7) and added unit tests; one test initially failed because the toy sign-election example summed to exactly zero (a degenerate tie), which we corrected to a nonzero-sum case. The suite then passed 17/17 offline. Results were written to `m2_results.md` and the README bar table updated, then committed and pushed.

A.15 Phase 13, M3: heterogeneous rank, and the code-transport saga (again)

For the structural carve-out we built a genuinely mixed-rank zoo by SVD-truncating the eight rank-16 adapters to ranks $\{4, 8, 16\}$ (free, no training). Running M3 reprised the transport problem in a sharper form (Table 12): the GitHub repository had *reverted to private* on its own, so the anonymous clone failed (v1); a clone-retry guard could not fix an auth failure (v2); embedding the package as a base64 tarball inside the notebook corrupted in transit (a $\sim$20 KB blob failed `zlib` decompression, v3); making the repository public again and installing via `git+` with a retry guard finally worked (v4). The result: OT-TIES merges the mixed-rank zoo at 0.698, only 1.0 point below the homogeneous case, and the GW operator runs at 0.623; both merges are pure CPU (264 to 359 s). This is the capability the rank-fixed baselines cannot offer at all.

Table 12: The M3 code-transport saga (getting our own package onto the worker, a second time).

ver	symptom	cause / fix
v1	`git clone` exited 128 ⇒ `ModuleNotFoundError: ot_lora_merge`	the repo had reverted to private; anonymous clone fails
v2	same, despite a clone-retry guard	a retry cannot fix an authentication failure
v3	embedded base64 tarball: `zlib error: invalid code lengths set`	a $\sim$20 KB blob inlined in the notebook corrupts in transit; do not embed code this way
v4	success	set the repo public (`gh repo edit --visibility public`) and `pip install git+...` with a 5× retry

A.16 Operational lessons (tooling pitfalls worth knowing)

Several non-research pitfalls cost real time and are worth recording for anyone driving Kaggle programmatically:

- **GPU sanity gate.** Put a tiny CUDA op (embedding + matmul + conv) at the very top of every GPU kernel and abort on failure; this turned the multi-minute "install then crash on first forward" loop of the environment saga into a few-second fail.
- **Status parsing.** A status watcher that greps for the bare substring "error" or "running" false-positives on transient CLI messages; match the structured token `KernelWorkerStatus.<STATE>` instead. We hit exactly this false "ERROR" once and wasted a check.
- **Signed download URLs.** The hub's signed result-file URLs must be pasted verbatim into a variable; manually retyping or line-wrapping them corrupts the signature and yields a 0-byte download.

- **Notebook title collisions.** Re-saving a kernel with a title already in use is rejected; bump the title ("...v2") and note that the live slug then becomes the bumped form.
- **Incremental dumps.** Writing the running log to disk after every step, and wrapping each evaluation in `try/except`, meant that even runs that ended in an error still yielded all results computed before the failure.

A.17 Where the user steered the project (decision forks)

For honesty about how the project was directed, the explicit decision points were: (i) the backbone choice (B/32 KnOTS over B/16, for bar comparability, Phase A.5); (ii) the order after M0 (tune the baselines to reproduce the published bar before proceeding); (iii) the response to the M1/M2 negative result (try a new OT formulation before falling back to the carve-out, Phase A.12); (iv) the code-transport policy (make the repository public for clean reproducibility, Phase A.10). Everything else followed from the data.

A.18 Kernel-by-kernel timeline

Table 13 lists every Kaggle kernel in order, whether it used the GPU, its purpose, and its outcome. The five probes and the early failed versions are GPU-off or fail-fast, which is why the total GPU consumption stayed inside the free weekly quota.

A.19 The exact free-compute recipe

A reader can reproduce every result with the following recipe.

1. **Offline**: `pip install -e .`; `pytest -q` (expect 17/17); the merge core needs only NumPy + POT.
2. **Make the code reachable**: push the repository to a public Git host; on the worker, `pip install git+<url>` (with a few retries to absorb transient clone failures). Do not embed the code as a base64 blob; do not rely on a private repo (anonymous clone fails).
3. **Environment on the GPU worker** (the one combination that works on a P100): `pip install torch==2.6.0 torchvision==0.21.0 --index-url https://download.pytorch.org/whl/cu124`, then `POT==0.9.6 peft transformers safetensors git+<fusion_bench>`, constrained to that torch. Run a CUDA sanity check first and abort on failure.
4. **Modelpool**: a `CLIPVisionModelPool` YAML over the eight `hoffman-lab/KnOTS-ViT-B-32_lora_R16_<task>` adapters; `merge_and_unload:true` for the baseline path, `false` (factored) for the OT path.
5. **Taskpool**: build the `CLIPVisionModelTaskPool` directly from FusionBench's `_template` (base `openai/clip-vit-base-patch32`; test datasets from the shipped per-task configs); do not rely on Hydra `compose`.
6. **Baselines**: `TaskArithmeticAlgorithm(scaling_factor=0.1)`; `TiesMergingAlgorithm(scaling_factor=0.3, threshold=20, remove_keys=[], merge_func='mean')`.
7. **Our method**: `ot_lora_merge.ot_ties_model(layer_adapters, k=128, keep=0.2, lam=0.7)` on the unmerged factors; add the returned per-layer $\Delta W^\star$ to the base $\{q, k, v, \text{out}\}_{\text{proj}}$ weights; evaluate.
8. **Heterogeneous rank**: SVD-truncate adapters to the desired ranks before the merge; the same `ot_ties_model` call works unchanged.

Table 13: Every Kaggle kernel, in order. "GPU" = whether the GPU was enabled; probes are GPU-off by design.

#	kernel (purpose)	GPU	outcome
1	probe 1, FusionBench API + bundled config	no	bundled pool is B/16; B/32 `tanganke` LoRA repos absent
2	probe 2, B/16 adapter internals	no	$r16$, $\alpha32$, q, v; parser validated
3	probe 3, KnOTS B/32 collection	no	$r16$, $\alpha16$, q, k, v, out, null base, `.bin`
4	probe 4, KnOTS loader + parser	no	loads natively; 48 layers extracted
5	probe 5, FusionBench Hydra/method API	no	taskpool from `_template`; method arg defaults
6	M0 v1, baselines	yes	taskpool + method-arg errors (fail-fast)
7	M0 v2, baselines	yes	CUDA `sm_60` missing (fail-fast)
8	M0 v3, baselines	yes	same; "preinstalled" torch is the broken one
9	M0 v4, baselines	yes	P100 ok but `.bin` load blocked (needs torch≥2.6)
10	M0 v5, baselines	yes	torch 2.6.0+cu124 works; TA/TIES key mismatch
11	M0 v6, baselines	yes	all baselines run; floor ≈ 0.64
12	M0 tuning sweep	yes	TAλ0.1=0.639 ≈ published; harness validated
13	M1, OT averaging	yes	align 0.610, barycenter 0.622 (floor; negative)
14	M2 rank/scale sweep	yes	flat 0.62 to 0.63; rank hypothesis refuted
15	M2 OT-TIES vs SVD-TIES	yes	OT 0.696 ¿ SVD 0.682 at λ0.5
16	M2 tune (1/2-basis)	yes	OT ¿ SVD every λ; 2-sided worse
17	M2 OT-TIES peak grid	yes	peak 0.708 at $\lambda0.7, \kappa0.2$
18	M3 v1	yes	private-repo clone fail (fail-fast)
19	M3 v2	yes	same, retry cannot fix auth (fail-fast)
20	M3 v3	yes	embedded base64 corrupts (fail-fast)
21	M3 v4	yes	success; mixed-rank OT-TIES 0.698, GW 0.623

Every kernel writes an incremental log and a result CSV; all are released under `results/`.

A.20 What a replicator should expect to see

Running the recipe should reproduce: the specialist accuracies (Table 5); the naive floor ≈ 0.64 with Task Arithmetic at λ=0.1 matching 0.638; the OT-averaging plateau at ≈ 0.62 across ranks; SVD-TIES≈ 0.687 and OT-TIES≈ 0.708 with the OT-over-SVD gap positive at every λ; and the mixed-rank OT-TIES at ≈ 0.698. Small (±0.005) deviations are expected from dataloader and BLAS nondeterminism; the *relative* orderings (OT > SVD > floor; mixed ≈ homogeneous) are stable.

B Extended primer on optimal transport

We collect the OT background for self-containedness. Given probability vectors $\boldsymbol{p} \in \Delta^a$, $\boldsymbol{q} \in \Delta^b$ and a ground cost $C \in \mathbb{R}^{a\times b}$, the Kantorovich problem is the linear program $\min_{P\in\Pi(\boldsymbol{p},\boldsymbol{q})}\langle P, C\rangle$ over the transport polytope $\Pi(\boldsymbol{p},\boldsymbol{q}) = \{P \geq 0 : P\mathbf{1} = \boldsymbol{p}, P^\top\mathbf{1} = \boldsymbol{q}\}$; its optimum is the (discrete) Wasserstein cost. The Monge problem is the deterministic special case where P is induced by a map;

it need not have a solution when masses are unequal, which is one reason Kantorovich's relaxation is used. The dual is $\max_{\boldsymbol{f},\boldsymbol{g}} \langle \boldsymbol{f}, \boldsymbol{p} \rangle + \langle \boldsymbol{g}, \boldsymbol{q} \rangle$ subject to $f_i + g_j \le C_{ij}$; the dual potentials are what Sinkhorn's iterates converge to (in log domain). Entropic regularization adds $-\varepsilon H(P)$, making the problem strongly convex and solvable by the matrix-scaling iteration of Section 3; as $\varepsilon \to 0$ the entropic optimum tends to the maximum-entropy exact optimum. The squared 2-Wasserstein distance $W_2^2(\mu, \nu)$ uses $C_{ij} = \|x_i - y_j\|^2$; its barycenter $\arg\min_\nu \sum_t \lambda_t W_2^2(\nu, \mu_t)$ [1] generalizes the Euclidean mean to distributions and, for fixed support size, is computed by the alternating transport/update scheme we use. Gromov–Wasserstein [17] replaces the cross-space cost by a quadratic mismatch of *intra*-space distance matrices (Eq. (5)); it is invariant to isometries of each space and therefore couples distributions that live in spaces of different dimension, the property we exploit for heterogeneous-rank adapters. For all problems in this project the supports are tiny (rank $r \le 16$, or $k \le 128$ barycenter atoms), so even the exact LP/EMD solver is negligible; entropic Sinkhorn is used for smoothness and speed. We use the POT library [10] for the solvers.

C Full algorithms

C.1 Wasserstein barycenter of direction distributions

Algorithm 2 Free-support Wasserstein barycenter (M-bary core)

Require: direction distributions $\{(U_t, \Sigma_t, V_t, \boldsymbol{p}_t)\}_{t=1}^T$; weights $\{\lambda_t\}$; target atoms R; cost C; reg. ε; iterations N

1: init $\nu = (U_\nu, \Sigma_\nu, V_\nu)$ from thin SVD of the stacked $[\lambda_t U_t \Sigma_t]$, $[V_t]$ (no dense product)
2: **for** $i = 1, \dots, N$ **do**
3: **for** $t = 1, \dots, T$ **do**
4: $C_t \leftarrow \textsc{Cost}(U_\nu, V_\nu, U_t, V_t)$ ▷ $R \times r_t$
5: $P_t \leftarrow \textsc{Sinkhorn}(\boldsymbol{p}_\nu, \boldsymbol{p}_t, C_t, \varepsilon)$
6: map task t into the ν frame via barycentric projection of P_t (transported mass)
7: **end for**
8: re-estimate $(U_\nu, \Sigma_\nu, V_\nu)$ as the top-R thin SVD of the accumulated factors
9: **break** if $\sum_k \sigma_{\nu,k}$ converged
10: **end for**
11: **return** dense $\Delta W_{\text{bar}} = U_\nu \Sigma_\nu V_\nu^\top$

C.2 TIES combiner (subspace coordinates)

Given per-task coordinate blocks $\{S_t \in \mathbb{R}^{k \times n}\}$, keep κ, scale λ: stack to $S \in \mathbb{R}^{T \times k \times n}$; trim each task to its top-κ fraction by $|\cdot|$ (per-task quantile threshold); elect sign $\gamma = \text{sign}(\sum_t S_t)$; form the disjoint mean over tasks agreeing with γ (denominator clamped at 1); scale by λ. Returns $S^\star \in \mathbb{R}^{k \times n}$.

C.3 SVD-TIES control and the model-level driver

The control SVD-TIES is Algorithm 1 with line 2 to 3 replaced by $U \leftarrow \textsc{TopLeftSingular}\big([\, s_1 B_1 \,|\, \cdots \,|\, s_T B_T \,],\, k\big)$ i.e. the leading left singular subspace of the column-stacked task up-projections (the KnOTS-style algebraic basis). Everything else, projection, TIES, reconstruction, is byte-for-byte identical, which is what makes the comparison clean. The model-level driver simply applies the chosen layer routine to every adapted layer:

Algorithm 3 Model-level merge (OT-TIES or SVD-TIES)

Require: per-layer adapter sets $\{\mathcal{A}^{(\ell)}\}_{\ell=1}^{L}$; base weights $\{W_0^{(\ell)}\}$; hyperparameters $(k, \kappa, \lambda, C, \varepsilon)$

1: **for** $\ell = 1, \ldots, L$ **do**
2: $\quad \Delta W^{\star(\ell)} \leftarrow \textsc{LayerMerge}(\mathcal{A}^{(\ell)}; k, \kappa, \lambda, C, \varepsilon)$
3: $\quad W^{(\ell)} \leftarrow W_0^{(\ell)} + \Delta W^{\star(\ell)}$
4: **end for**
5: **return** merged model $\{W^{(\ell)}\}$

C.4 Gromov–Wasserstein heterogeneous-rank merge

Choose anchor $a = \arg\max_t r_t$. Compute intra-task structures $D^{(t)}$. For each $t \neq a$, solve the entropic GW coupling P^{at} between $(D^{(a)}, \boldsymbol{p}_a)$ and $(D^{(t)}, \boldsymbol{p}_t)$ (Eq. (5)), map task t into the anchor frame via the barycentric projection of P^{at}, and accumulate the λ_t-weighted transported update; refactor to the target rank.

C.5 Identity guarantee

Lemma 2 (Single-adapter identity)**.** *For $T = 1$, every merge operator in this project (M-align, M-bary,* OT-TIES*, GW) returns exactly the input update ΔW_1.*

Proof. With one task the alignment/coupling is to the task itself: the anchor is task 1, the transport plan is $\mathrm{diag}(\boldsymbol{p}_1)$, and the barycentric projection is the identity map, so the transported update equals ΔW_1. In OT-TIES, the barycenter of a single distribution is that distribution, its leading subspace U spans the column space of ΔW_1, and $UU^\top \Delta W_1 = \Delta W_1$ since ΔW_1's columns lie in range(U); TIES with a single task elects that task's signs and returns it scaled by λ (we set $\lambda = 1$ for the identity check). Hence the output is ΔW_1. □

This identity is one of the unit tests in the released package and is what lets us trust the multi-task path: any deviation from identity at $T = 1$ would signal a bookkeeping bug in the transport or reconstruction.

D Detailed experimental protocol

Adapters and base. The eight adapters are `hoffman-lab/KnOTS-ViT-B-32_lora_R16_{sun397, stanford_cars, resisc45, eurosat, svhn, gtsrb, mnist, dtd}`, each rank $r = 16$, $\alpha = 16$ (scale $s = 1$), targeting the four attention projections $\{q, k, v, \mathrm{out}\}_{\mathrm{proj}}$ of every transformer block of the CLIP ViT-B/32 vision encoder [25]; the backbone is `openai/clip-vit-base-patch32`. Loading each adapter unmerged yields, per block, four LoRA pairs, i.e. 24 adapted linear maps over 12 blocks (48 factor matrices) at $m{=}n{=}768$.

Building the merge inputs. For the baseline path we fold each LoRA into the full weight (`merge_and_unload`) so that FusionBench's task-arithmetic/TIES implementations operate on full weights, matching how the published numbers were computed. For our method we load the adapters *unmerged* and read the factors A_t, B_t directly, since OT-TIES needs the factored ΔW_t. The scale $s = \alpha/r = 1$ means no extra rescaling is applied.

Evaluation. We use FusionBench's CLIP vision task pool [29], built from its shipped 8-task configuration with the CLIP backbone and processor overridden to ViT-B/32. For each task the text encoder produces zero-shot classification weights from the class names and prompt templates; the merged vision encoder produces image features; accuracy is top-1 over the task's test split. Normalized accuracy divides by the specialist's accuracy (Table 5), and we report the unweighted mean over the eight tasks (Eq. (2)). Batch size 128, no test-time augmentation, deterministic.

Applying the merged update. OT-TIES returns a dense per-layer $\Delta W^\star = US^\star$ of rank up to k; we add it directly to the corresponding base $\{q, k, v, \text{out}\}_{\text{proj}}$ weight (it generally exceeds rank 16, so it is *not* re-folded into a rank-16 LoRA). For the GW path we reconstruct $\Delta W^\star = B^\star A^\star$ from the refactored output and add it likewise.

E Implementation and hyperparameters

- **Subspace dim** $k = 128$ (the full $T \cdot r = 8 \times 16$); **keep** $\kappa = 0.2$; **scale** $\lambda = 0.7$ (OT-TIES defaults, the empirical peak).
- **Ground cost** paired cosine (Eq. (6)); **Sinkhorn** $\varepsilon = 10^{-2}$ with an exact-EMD fallback on numerical underflow; barycenter ≤ 12 iterations.
- **Baseline configs** (FusionBench): Task Arithmetic $\lambda = 0.1$; TIES disjoint-mean $\lambda = 0.3$, threshold 20. Specialist accuracies in Table 5.
- **Reconstruction** one-sided: $\Delta W^\star = US^\star$ added directly to the base $\{q, k, v, \text{out}\}_{\text{proj}}$ weights (the merged update can exceed rank 16, so it is applied as a full-rank-up-to-k delta, not re-folded into a rank-16 LoRA).
- **Environment** `torch==2.6.0+cu124`, FusionBench from source, POT 0.9.6, NumPy core; 17/17 unit tests pass offline; GPU used only for evaluation.

F Per-task accuracies

Table 14 gives per-task accuracies for the naive baselines and the OT-average merges (the configurations for which full per-task vectors were logged); the headline OT-TIES per-task breakdowns and all sweep cells are in the released CSVs (`results/m0_baselines.csv`, `m1_results.csv`, `m2tune.csv`, `m2peak.csv`, `m3_hetero.csv`).

Table 14: Per-task accuracy (not normalized) for representative configurations. The interference signature is visible: conflict-heavy SVHN/GTSRB/EuroSAT collapse under averaging.

Method	SUN397	Cars	RESISC45	EuroSAT	SVHN	GTSRB	MNIST	DTD	avg
Simple average	0.623	0.609	0.628	0.455	0.427	0.405	0.531	0.424	0.513
Task Arith. λ=0.1	0.628	0.609	0.633	0.483	0.405	0.392	0.535	0.431	0.514
TIES mean λ=0.3	0.627	0.612	0.614	0.495	0.405	0.340	0.563	0.428	0.510
OT align	0.627	0.615	0.613	0.431	0.341	0.342	0.491	0.431	0.486
OT barycenter	0.626	0.607	0.617	0.460	0.378	0.353	0.520	0.428	0.499

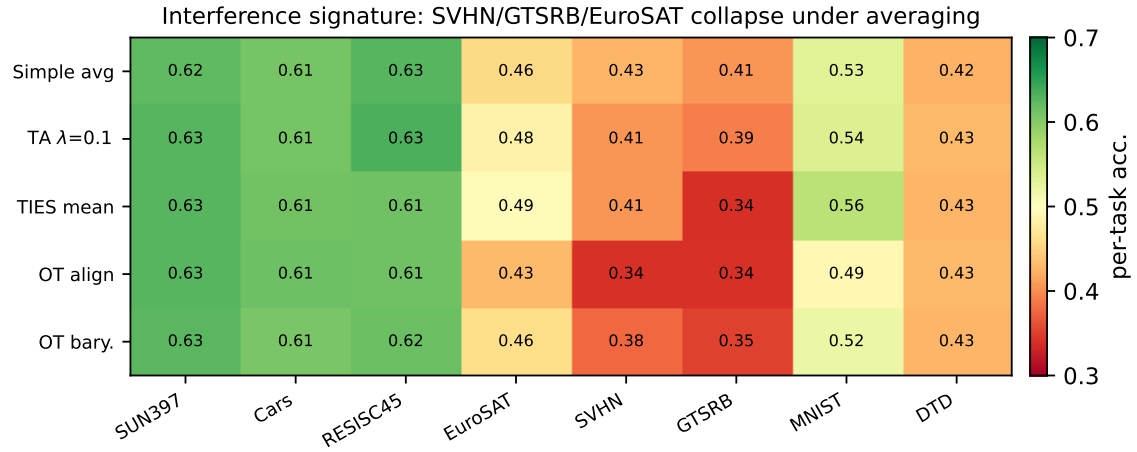

Figure 5: Per-task accuracy heatmap for the averaging-family configurations. The interference signature is stark: near-orthogonal tasks (SUN397, Cars, RESISC45) stay green, while the conflict-heavy SVHN/GTSRB/EuroSAT collapse to red under any averaging combine. This is the failure mode Proposition 1 predicts and that the TIES sign-election in OT-TIES repairs.

G Reproducibility case study: a research project on free compute

Because compute-frugality is part of our thesis, we document the actual free-tier workflow, including the failures, as a reproducibility artifact. The entire study ran on Kaggle's free notebooks; the GPU was used only to evaluate merged models, never to merge. Several environment pitfalls are worth recording because they will bite any reader reproducing on the same free tier:

- **CUDA/torch matrix.** The default free image ships a CUDA build whose kernels drop the Pascal (`sm_60`) architecture used by the assigned P100, producing "no kernel image available"; pinning an *older* build then trips `transformers`' refusal to `torch.load` legacy `.bin` weights under a CVE guard. The unique working point is `torch==2.6.0+cu124`.
- **Cheap probes before GPU spend.** Every uncertainty about the external API (adapter repo IDs, PEFT key layout, FusionBench taskpool construction, method constructor signatures) was resolved by *no-GPU* probe kernels before any GPU run, so the first GPU run was already correct in its data path.
- **Fail-fast scaffolding.** Each run writes incremental logs and wraps every evaluation in a try/except so a single failed cell never discards the whole run's results.

The net effect is that a multi-week research project, five baseline configurations, two negative OT families, the OT-TIES ablation grid, a peak grid, and a heterogeneous-rank study, fit comfortably inside the free weekly quota.

H Catalogue of negative and null results

We list the configurations that did *not* work, since they delimit the contribution. (1) OT alignment and Wasserstein barycenter as the *final* combine: at the floor for all ranks $\{16, 32, 64, 128\}$ and all scalings (Table 7). (2) Increasing the barycenter scaling $\lambda > 1$: monotone degradation. (3)

Two-sided (U and V) reconstruction: worse than one-sided for both SVD and OT bases (over-compression; Table 8). (4) Large TIES keep fraction ($\kappa = 0.3$) or small ($\kappa = 0.1$): both below $\kappa = 0.2$ (Table 9). These nulls are consistent with the single mechanism we identify: interference resolution is the dominant lever and the OT subspace is a second-order (but consistent) improvement on top of it.

I Complete sweep tables

For completeness we give the full grids underlying the summary tables in the main text; all are released as CSVs.

Table 15: Full Task-Arithmetic and TIES tuning (M0). Average normalized accuracy.

Method	config	NA
Simple average	n/a	0.636
Task Arithmetic	$\lambda = 0.1$	0.639
Task Arithmetic	$\lambda = 0.2$	0.566
Task Arithmetic	$\lambda = 0.3$	0.405
Task Arithmetic	$\lambda = 0.4$	0.239
TIES (merge=mean)	$\lambda = 0.3$, $\tau = 20$	0.634
TIES (merge=mean)	$\lambda = 1.0$, $\tau = 20$	0.454

Table 16: Full OT-TIES/SVD-TIES grid. Left: the initial λ sweep at $\kappa = 0.2$, one-sided, plus the two-sided variant. Right: the OT-TIES peak grid over (λ, κ). NA = average normalized accuracy.

config	SVD-TIES	OT-TIES
λ=0.3 (1-side)	0.658	0.665
λ=0.4 (1-side)	0.672	0.682
λ=0.5 (1-side)	0.682	0.696
λ=0.6 (1-side)	0.687	0.706
λ=0.5 (2-side)	0.680	0.654

OT-TIES	κ=0.1	κ=0.2	κ=0.3
λ=0.6	n/a	0.706	0.693
λ=0.7	0.701	**0.708**	0.698
λ=0.8	0.667	0.703	0.690

Table 17: OT-averaging family (M1/M2), full. Both M-align and M-bary stay at the floor across ranks and scalings.

Method	config	NA
M-align	paired, ε=10^{-2}	0.610
M-bary	R=16	0.622
M-bary	R=32	0.625
M-bary	R=64	0.627
M-bary	R=128	0.626
M-bary	R=128, λ=2	0.556
M-bary	R=128, λ=4	0.263

J Reproducibility checklist

- **Code**: released; the merge core is pure NumPy (`ot_lora_merge`), import-light (does not pull torch), 17/17 unit tests.
- **Adapters**: public, `hoffman-lab/KnOTS-ViT-B-32_lora_R16_<task>` (8 tasks).
- **Data**: the eight FusionBench test sets, all openly available.
- **Compute**: free Kaggle (T4/P100); merges are CPU; one-command notebooks released.
- **Logs**: every run's stdout and result CSV is in `results/`.
- **Determinism**: canonical SVD sign-fixing; fixed Sinkhorn ε and iteration caps.

K Assets, licensing, and ethics

All assets are open. The backbone CLIP ViT-B/32 [25] and the KnOTS adapters are publicly released for research use; the eight evaluation datasets (SUN397, Stanford Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD) are standard research benchmarks accessed through the FusionBench [29] loaders; the OT solver is the open POT library [10]. Our own code is released under a permissive license, contains no trained model weights (only the data-free merge code and configs), and pulls all third-party assets at runtime from their official sources. The method operates exclusively on already-published adapters and introduces no new data collection. Because it lowers the cost of multi-task model reuse without training, its broader-impact profile matches that of public adapter sharing generally; we are not aware of a misuse vector specific to choosing a merge subspace by transport rather than by SVD.

L Glossary

Adapter / LoRA a low-rank additive update $\Delta W = sBA$ to a frozen layer.

Task vector the (vectorized) update of a fine-tuned model relative to the base.

Interference destructive overlap of independently trained updates (redundancy, sign conflict, magnitude disparity).

Interference resolution a combiner that trims, sign-elects, and disjoint-means (TIES) rather than averaging.

Shared subspace a common basis U in which all tasks' updates are expressed before combining.

Wasserstein barycenter the distribution minimizing the weighted sum of squared transport distances to a family of distributions; here, of the tasks' singular-direction distributions.

Gromov–Wasserstein a transport that couples distributions across incomparable spaces by matching intra-space geometry; here, what enables heterogeneous-rank merging.

Normalized accuracy merged accuracy divided by specialist accuracy, averaged over tasks.